

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ ГУМАНИТАРНЫЙ
УНИВЕРСИТЕТ»
(ФГБОУ ВО «РГГУ»)
ИНСТИТУТ ЛИНГВИСТИКИ

Олейникова Елизавета Олеговна

**Решение задач классификации текста при помощи языковой модели BERT
с добавлением семантической информации на базе формата CoBaLD.**

Курсовая работа студентки 3-го курса
очной формы обучения

Направление подготовки
45.03.03 «Фундаментальная и прикладная лингвистика»

Направленность (профиль)
«Фундаментальная и прикладная лингвистика»

Директор Института лингвистики
Доктор филол. наук

_____ Я. В. Тестелец
«__» _____ 20__ г.

Научный руководитель
преподаватель УНЦ компьютерной
лингвистики

_____ А. М. Ивойлова
«__» _____ 20__ г.

Москва 2025

Оглавление

Введение.....	4
Машинное обучение.....	5
Способы представления текстов в машиночитаемом виде.....	5
Контекстные векторы.....	6
Динамические векторы.....	8
Форматы разметки.....	16
FrameNet.....	16
Prague Textogrammatical Graph (PTG).....	17
UCCA.....	17
AMR.....	18
СемЭТАП.....	18
CoBaLD.....	19
Задачи.....	20
Анализ тональности.....	20
Детекция токсичности.....	22
Обогащение эмбедингов семантикой.....	23
Практическая часть.....	26
Датасеты.....	26
Метрики в машинном обучении.....	27
Ход работы.....	28
Результаты.....	29
Детекция токсичности.....	29
Анализ тональности.....	32
Заключение.....	38
Список литературы.....	39

Введение

Данная работа посвящена исследованию применимости семантической разметки в формате CoBaLD [Petrova et al. 2023] для обогащения эмбедингов моделей класса BERT эксплицитной семантической информацией. Исследование будет проводиться для задач классификации текстов, а именно анализа тональности и детекции токсичности. Семантическая разметка, используемая в нашей работе, содержит информацию о семантических ролях и лексических значениях слов. Эмбединги в модели BERT являются машиночитаемым контекстозависимым представлением слов.

Гипотеза

Наша гипотеза состоит в следующем:

1. Обогащение предобученных эмбедингов BERT семантической информацией, структура которой задана человеком, поможет улучшить качество решения моделью задач анализа тональности и детекции токсичности;
2. Формат разметки CoBaLD действительно отражает реальные языковые явления и пригоден для обогащения языковых моделей семантикой.

Актуальность

Наше исследование может иметь практическую пользу для улучшения качества работы языковых моделей. Кроме того, наше исследование поможет понять, насколько семантическая разметка CoBaLD пригодна для обогащения предобученных эмбедингов.

Новизна

Качество работы больших языковых моделей на последние годы достигло очень высоких показателей. Однако, есть задачи, общепринятого решения которых нет до сих пор. Например, работа над дизамбигуацией значений слов (снятием омонимии) и тем, что языковые модели игнорируют отрицание. Причина этих проблем – то, как языковые модели строят эмбединги – числовые представления токенов. Результаты нашей работы могут стать вкладом в понимание того, может ли добавление внешней информации, формат которой определен человеком, улучшить качество представления слов в языковых моделях.

Цель

Определить, влияет ли добавление к эмбедингам предобученных языковых моделей семейства BERT эксплицитной семантической информации на качество выполнения задач классификации текстов.

Задачи

1. Написать бейзлайны¹ для решения выбранных задач без использования семантики.
2. Выполнить автоматическую разметку датасетов в формате CoBaLD с использованием обученного на данных русского языка парсера [Баяк 2024].
3. Добавить к предобученным эмбедингам модели BERT семантическую информацию из разметки при помощи конкатенации эмбедингов.
4. Проанализировать и сравнить результаты работы бейзлайнов до и после добавления семантической разметки.

¹ Бейзлайн (baseline) – базовая, простейшая модель, которая нужна для дальнейшей оценки решений выбранной задачи.

Машинное обучение

Идея о том, что компьютер может имитировать человеческую речь появились в 1940-1950 гг. Развитие NLP (Natural language processing) в то время можно разделить на два направления: правилное или логическое, и статистическое. Логическому направлению сопутствовало появление теории генеративной грамматики Н. Хомского [Chomsky N., 1956]. В основе статистического метода лежали такие понятия, как цепи Маркова, Байесовский вывод, энтропия. Статистический и правилный направления существовали параллельно до начала 2000-ых годов, когда стали доступны большие массивы языковых данных: Penn Treebank [Marcus et al., 1993], Prague Dependency Treebank [Hajic, 1998], PropBank [Palmer et al., 2005] и некоторые другие. В этот же период возросли вычислительные мощности компьютеров. Два этих фактора способствовали развитию машинного обучения, которое вытеснило правилые алгоритмы и простые статистические модели. [Jurafsky D., Martin J. H., 2008]

Правилые алгоритмы требовали объемных и сложно устроенных схем действий и при этом показывали сравнительно низкое качество. В отличие от правилых алгоритмов, алгоритмы, в которых применяется машинное обучение, выстраивают некую математическую модель на основе тренировочных данных, и при помощи этой модели могут строить предсказания для неизвестных данных.

В основе алгоритмов машинного обучения лежат линейная алгебра, математический анализ и статистика. Суть этих алгоритмов состоит в том, чтобы преобразовать входные данные таким образом, чтобы на выходе получить результат, соответствующий ожидаемому. Разумеется, эти преобразования должны работать как минимум для большей части обучающих данных. Любой алгоритм машинного обучения, таким образом, представляет собой решающую функцию с множеством переменных (весов), которая устанавливает зависимость между ответами и признаками в данных (например, зависимость между ценой квартиры и ее числовыми характеристиками, такими, как площадь). Коэффициенты для каждой переменной подбираются при помощи функции потерь, которая показывает, насколько результат применения функции далек от ожидаемого. Собственно процесс подбора оптимальных коэффициентов для решающей функции и называется обучением, а данные, на которых оно происходит – тренировочным датасетом.

Нейронные сети - это подкласс методов машинного обучения, который стал популярен с 2010 гг. Отличие его от других методов машинного обучения состоит в том, что признаки, которые будут использоваться на обучении и в работе модели, не нужно задавать эксплицитно. Модель сама извлекает признаки, для которых будут обучаться веса. Это, с одной стороны, улучшило результаты применения нейронных моделей по сравнению с другими, с другой стороны, требовало большого количества обучающих данных и вычислительных ресурсов, которые стали доступны к 2010-ым. [LeCun Y. et al. 2015]

Способы представления текстов в машиночитаемом виде

Для того, чтобы преобразовать текстовые данные в какой-либо машиночитаемый формат, их требуется сегментировать. Типы элементов, на которые будет проходить сегментация, зависит от типа задач. В общем случае, сегментами на более крупном уровне выступают предложения, а на базовом уровне - токены – значимая для анализа последовательность символов. Как правило, токенами считаются слова в бытовом понимании этого термина, а также пунктуация. Что именно будет считаться словом в конкретном случае зависит от типа задач.

Самый простой способ представить токены в машиночитаемом виде – это вектры one-hot encoding [LeCun Y. et al., 2015]. В этом способе кодирования каждому токenu соответствует вектор, имеющий длину равную длине словаря. В этих векторах на месте, имеющем индекс, равный индексу токена в словаре, стоит 1, а на всех остальных позициях – 0.

Этот подход имеет ряд недостатков: во-первых, текст представляет собой высоко разреженное векторное пространство большой размерности, причем размерность зависит от размера словаря. Во-вторых, такой способ кодирования токенов не учитывает связь между ними. [LeCun Y. et al., 2015]

С one-hot векторами тесно связана модель bag-of-words (BoW) [Manning C. D., 2009]. В данной модели вектором текста является вектор, где каждое число представляет собой частоту конкретного токена в используемом датасете. Этот метод все еще не учитывает связи между токенами и порядок слов в предложении, а также словоизменение (хотя есть усовершенствованные варианты BoW где эта проблема решена).

Следующий метод – TF-IDF учитывает не только частоту слов в тексте, но и их важность [Spärck Jones K. 2004]. Идея важности состоит в том, что некоторые слова встречаются в текстах очень часто, однако они примерно одинаково частотны для всех текстов, и поэтому они не очень важны для представления конкретного текста. Чтобы это учесть при представлении предложения в виде вектора, вес слов определяется не только их частотностью, но и встречаемостью в текстах.

$$TF - IDF(t, d) = TF(t, d) \times IDF(t)$$

где t – слово, d – документ, TF – частота слова t в документе d , а

$$IDF(t) = \log \frac{N}{DF(t)},$$

где N – количество документов в корпусе, а $DF(t)$ – количество документов, в которых встречается слово t .

Наибольшая мера TF-IDF будет у слов, которые встречаются в небольшом числе документов, при этом имеют высокую частотность.

Контекстные векторы

Большим недостатком вышеупомянутых методов представления текстов является то, что они не учитывают контекст слова. Они дают нам общие статистические данные про слова в целом, но как правило, мы имеем дело со связными текстами, где семантика слов варьируется в зависимости от контекста.

N-gram модели - это это первые статистические языковые модели, призванные смоделировать поведение естественного языка и для этого вычислить вероятностное распределение слов в нем [Jurafsky D., Martin J. H. 2025]. Очевидно, что для моделирования естественного языка необходимо учитывать контекст; для этого в N-грамных моделях выбирается окно из n слов, в которое попадает искомое слово и его левый контекст. На большом корпусе данных можно собрать все контексты, в которых употребляется слово, а также их частоту. Собрав эту информацию для всех слов в корпусе, мы можем использовать ее для предсказания следующего слова по предыдущим n словам. Слова в тексте рассматриваются как последовательность, обладающая марковским свойством отсутствия памяти: то есть, при расчете вероятности текущего токена учитывается только n предыдущих токенов, но не вся

последовательность с начала до текущего момента. Тогда искомое слово вычисляется путем подсчета вероятности появления цепочек из $n-1$ слов из контекста и каждого слова, которое встречалось в корпусе в данном контексте. Результатом является слово, которое оказалось в цепочке с наибольшей вероятностью.

Этот способ имеет ту же проблему, что и one-hot encoding: векторы слов будут иметь большую размерность, при этом большая часть векторов будет состоять из нулей. К тому же, для слов, которые редко встречаются в тексте, векторы будут содержать мало информации. Проблема с высокой размерностью векторов слов называется “проклятием размерности”. Она актуальна не только для NLP, но и для многих других областей машинного обучения. Требуется способ представления слов, который поможет обобщить семантическую и синтаксическую информацию о них, и при этом будет иметь низкую размерность и разреженность.

Word2Vec

Данный метод, как и предыдущий, опирается на контекст слова, но работает уже на нейронных сетях, что значительно повышает его качество [Mikolov T. et al., 2013]. Модель Word2vec имеет три слоя. На верхний, подаются тексты в виде набора one-hot-encoding векторов для каждого токена. Далее на скрытом слое для каждого токена строятся векторы, которые будут обучаться в процессе тренировки модели. Изначально все они инициализируются случайными числами. Далее модели подается на вход датасет с пропущенными токенами, которые модель должна восстановить по контексту. Строго говоря, мы не можем сказать, что именно означает каждый элемент полученного вектора, поскольку модель возвращает их без каких-либо “пояснений”. На выход подаются векторы, имеющие длину, равную длине словаря данного текста, где каждое число - вероятность появления токена в заданном контексте.

Данный метод лежит в основе работы всех нейронных сетей, используемых для обработки естественного языка. Векторы слов в языковых моделях имеют свое название - **эмбединги**. Косинусное расстояние между эмбедингами показывает, насколько близки слова по семантике. Основой для представления токенов, а также семантических меток в нашем исследовании также являются эмбединги.

В моделях Word2Vec используется две стратегии предсказания слов по контексту: Skipgram и CBoW. В CBoW (Continuous Bag of Words) [Rong X. 2014] предсказывается слово по контексту, причем как правому, так и левому. Skipgram [Mikolov T. et al., 2013] подразумевает предсказание левого и правого контекстов по заданному слову.

GloVe

GloVe - это усовершенствованный, по сравнению с Word2Vec, метод представления слов в контексте в виде векторов [Pennington J. et al., 2014]. GloVe учитывает широкий контекст, не ограничиваясь узким контекстом в плавающем окне. При этом учитывается “важность” каждого слова из контекста искомого слова для построения его вектора. Чем дальше слово находится от искомого, чем меньший вес оно имеет при построении вектора.

Fasttext

Модель Fasttext также основана на Word2Vec, а конкретно на стратегии Skipgram с добавлением метода subwords – деления токенов на подслова [Bojanowski P. et al., 2017]. Это позволяет модели учитывать не только связь токенов, но и их структуру.

Метод деления токенов на подслова основан на N-gramm модели: слово представляется как сумма подслов, которые попали в окно длиной n символов. [Joulin A. et al., 2016]

Метод деления на подслова используется и в языковых моделях, основанных на нейронных сетях, в частности в модели BERT, которую мы используем в нашей работе. Это позволяет избежать проблем, связанных с определением того, что считается токеном, и позволяет модели самостоятельно выучить паттерны в словообразовании и словоизменении.

Модели Word2Vec, GloVe и fasttext, в отличие от других упомянутых выше алгоритмов, представляют собой нейронные сети. Они базируются на архитектуре MLP (Multilayer Perceptron) и представляют собой неглубокие (shallow) нейронные сети. Далее мы рассмотрим модели, основанные на глубоких нейронных сетях - DNN (Deep Neural Networks) [LeCun Y. et al., 2015]. Они имеют более сложную архитектуру и показывают лучшее качество. В частности, модель BERT, которую мы используем, является глубокой нейронной сетью.

Динамические векторы

Упомянутые выше модели – Word2Vec, GloVe и fasttext – на обучении строят статические векторы, которые не меняются в зависимости от конкретного контекста слова и, следовательно, не учитывают изменения семантики в зависимости от контекста и полисемии. В то же время, их преимуществом является экономичность ресурсов, затрачиваемых на работу и скорость.

Модели, которые будут рассмотрены ниже – это модели, в которых используются динамические эмбединги, способные меняться в зависимости от контекста слова. Они, в отличие от статических эмбедингов, учитывают полисемию и показывают более высокие результаты, хотя и требуют больше вычислительных мощностей.

Рекуррентные нейронные сети

Идея рекуррентных нейронных сетей состоит в том, чтобы обрабатывать последовательные данные не одновременно, а поочередно, при этом сохраняя информацию о всех предыдущих элементах последовательности [Zhang A. et al., 2023]. Такой способ обработки текстовой информации кажется интуитивным в аналогии с тем, как человек читает текст: каждое новое слово воспринимается вместе со всеми предыдущими словами, то есть в контексте.

На каждом шаге рекуррентная нейронная сеть получает на вход новый элемент последовательности, например, новый токен. Эмбединг этого токена перемножается с матрицей обучаемых весов и суммируется с эмбедингом состояния нейронной сети на предыдущем шаге, помноженным на другую матрицу обучаемых весов:

$$X_t W_{xh} + H_{t-1} W_{hh}, \text{ где}$$

t – номер шага,

X_t – входные данные на шаге t ,

W_{xh} – матрица весов для входных данных,

H_{t-1} – скрытое состояние нейронной сети на предыдущем шаге,

W_{hh} – матрица весов для скрытого состояния.

Функция от этой суммы представляет собой скрытое состояние нейронной сети на данном шаге и будет передано на следующий шаг нейронной сети.

$$H_t = f(X_t W_{xh} + H_{t-1} W_{hh} + b_h), \text{ где}$$

H_t – скрытое состояние сети на шаге t ,

f – функция активации (в случае с *BERT*, *GELU*) [Lee M., 2023]

b_h – смещение.

На выходе, когда вся последовательность обработана, получается эмбединг, который хранит информацию о всей последовательности.

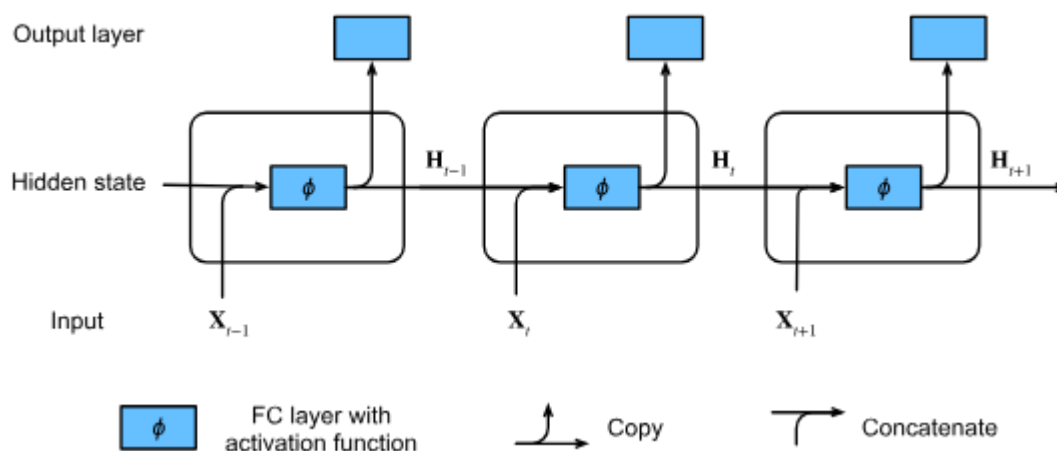


Рисунок 1. Архитектура RNN
[Zhang A. et al., 2023, Fig. 9.4.1]

Данный метод обработки текстов привнес важную идею в NLP, а именно учет влияния контекста на представление слова. Однако у него есть существенные недостатки: умножение множества слагаемых нестабильно, нет гарантии, что удастся сохранить зависимость вектора текста на выходе от первых его слов.

LSTM

LSTM (Long short-term memory) – это усовершенствованный вариант рекуррентных сетей, который решает проблему сохранения сигнала от первых элементов последовательности до последних [Hochreiter S., Schmidhuber J., 1996]. Для этого из каждой ячейки в сети в следующую ячейку передается не только скрытое состояние, но и состояние ячейки – последнее и позволяет не потерять сигнал по ходу последовательности.

Ячейкой памяти называется набор вычислений, которые проводятся на каждом шаге с определенным набором весов. Набор весов для каждого шага одинаков, при этом веса являются обучаемыми.

В каждую ячейку из предыдущей (если она не первая в последовательности) передается скрытое состояние (hidden state) и состояние ячейки (cell state). Ячейка имеет три типа матриц: матрица забывания (forget gate), матрица добавления данных (input gate) и матрица результата (output gate).

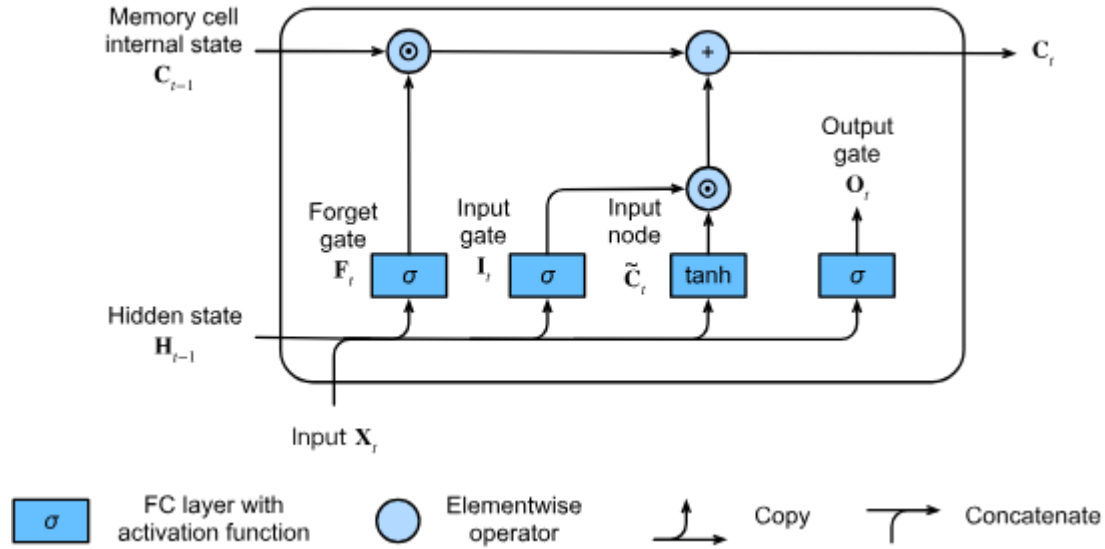


Рисунок 2. Архитектура LSTM
[Zhang A. et al., 2023, Fig. 10.1.3]

Сначала скрытое состояние предыдущей ячейки вместе с текущим входом перемножается с матрицей забывания и проходит через сигмоиду, которая определяет, какую часть скрытого состояния передать дальше, а какую “забыть”:

$$F_t = \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f), \text{ где}$$

F_t – выход из матрицы забывания,

W_{xf} – матрица весов забывания для входа,

W_{hf} – матрица весов забывания для скрытого состояния,

b_f – смещение матрицы забывания.

Далее скрытое состояние проходит еще через одну матрицу - матрицу добавления данных. Перед перемножением с матрицей весов, скрытое состояния пропускается через сигмоиду, которая решает, какие значения нужно обновить:

$$I_t = \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i), \text{ где}$$

I_t – выход из матрицы добавления данных на шаге t ,

W_{xf} – матрица весов добавления данных для входа,

W_{hi} – матрица весов добавления данных для скрытого состояния,

b_i – смещение матрицы добавления данных.

Далее эти значения, перемноженные на матрицу весов, проходят через тангенс:

$$C_t = \tanh(X_t W_{xc} + H_{t-1} W_{hc} + b_c), \text{ где}$$

C_t – матрица значений, на которые нужно обновить выбранные данные,

W_{xc} — матрица весов обновления данных для входа,
 W_{hc} — матрица весов обновления данных для скрытого состояния,
 b_c — смещение матрицы обновления данных.

Состояние ячейки памяти, которое будет передано в следующую ячейку, таким образом, представляет собой сумму выходов из матриц забывания и обновления весов:

$$C_t = F_t \odot C_{t-1} + I_t \odot \underline{C}_t, \text{ где}$$

C_t — состояние ячейки на шаге t ,

C_{t-1} — состояние ячейки на предыдущем шаге.

Следующим этапом нужно определить, что будет выдаваться на этом шаге (если нужно) и каким будет скрытое состояние. Для этого скрытое состояние проходит через сигмоиду, которая определяет, что будет выдаваться, а текущее значение ячейки проходит через тангенс. Скрытое состояние текущей ячейки представляет собой перемножение этих векторов.

$$H_t = O_t \odot \tanh(C_t), \text{ где}$$

H_t = скрытое состояние ячейки на текущем шаге t .

На рекуррентных нейронных сетях, а конкретно на bi-LSTM работает модель ELMO [Peters M. E. et al., 2018f]. Bi-LSTM представляет собой вариант LSTM, в котором вычисления идут в двух направлениях: из начала в конец и из конца в начало. Это позволяет модели использовать не только левый контекст, но и правый, что помогает улучшить качество предсказаний.

Входные данные в ELMO подаются в bi-LSTM не сразу, сначала они проходят через сверточную нейронную сеть (CNN), которая формирует эмбединги подслов (subwords). В отличие от упомянутого выше метода деления последовательности символов на подслова при помощи n-грамм, CNN не просто суммирует эмбединги частей слов, а выучивает паттерны словообразования, а также позволяет модели формировать эмбединги токенов, которые не встречались на обучении. Архитектура ELMO представляет собой не просто bi-LSTM с CNN: ELMO состоит из двух слоев bi-LSTM, где на второй слой подается выход из первого, а далее - на линейный слой. Это позволяет улавливать дальние, абстрактные связи в тексте.

Трансформеры

Рекуррентные нейронные сети, в том числе LSTM, действительно решили проблему того, как учитывать контекстно-зависимые значения слов. Однако вместе с этим сильно увеличились вычислительные и временные ресурсы, необходимые для обучения модели. Во-первых, LSTM предполагает хранение информации о всех предыдущих состояниях сети с самого начала. Даже с учетом того, что какие-то части информации “забываются”, на длинных последовательностях LSTM требует больших вычислительных мощностей. Во-вторых, в силу того, что вычисления проводятся поэлементно, они занимают много времени в сравнении с тем, как если бы они проводились для всех элементов одновременно.

Эту проблему решили нейронные сети с архитектурой трансформеров, лежащие в основе моделей, которые сейчас показывают наилучшие результаты [Vaswani A. et al., 2017]. В нашей работе мы используем модель, построенную на данной архитектуре.

Классические модели-трансформеры состоят из блоков энкодеров и декодеров. Первые принимают информацию на вход и возвращают ее в виде векторов с извлеченными из входных данных признаками, а вторые порождают выходные данные. В блоке энкодеров первый энкодер принимает входные данные, а все остальные принимают выходы из энкодера перед ними. Таким же образом устроен и блок декодеров. Энкодер и декодер внутри устроены схожим образом: векторы входных данных подаются в слой самовнимания, а выход из этого слоя передается в полносвязный слой с нормализацией. В декодере между слоем самовнимания и полносвязным слоем находится слой внимания, связывающий между собой энкодер и декодер.

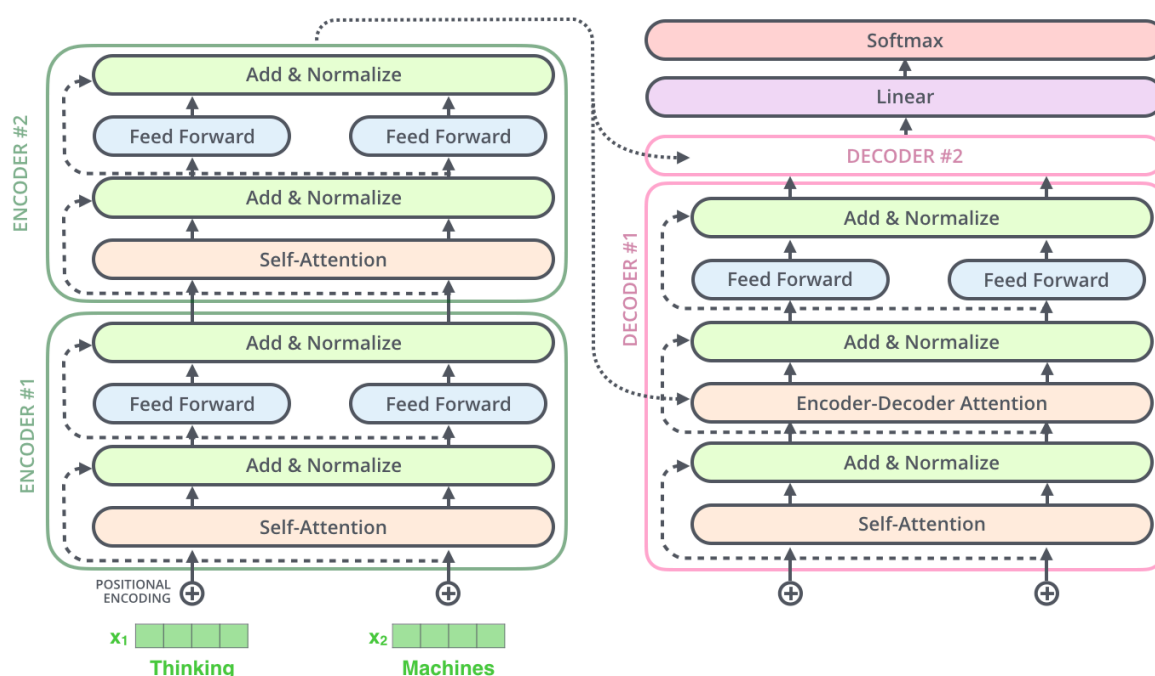


Рисунок 3. Архитектура трансформеров

Источник: Jay Allamar²

Важное отличие трансформеров от нейронных сетей предыдущих поколений - это то, что вся последовательность, например, токенов передается не поочередно, а параллельно, тем самым значительно уменьшается время, необходимое на их обработку. Кроме того, параллельная обработка всей последовательности токенов решает существенную проблему нейронных сетей, основанных на RNN: на выходе вся последовательность токенов представляется в виде одного вектора, из-за этого часть информации неизбежно теряется. В отличие от RNN, трансформеры не объединяют эмбединги всех токенов в один вектор, а хранят матрицу, состоящую из эмбедингов всех токенов.

После того, как последовательность входных данных попадает в нейронную сеть, она проходит через слой самовнимания. Механизм самовнимания - один из ключевых

² <https://jalamar.github.io/illustrated-transformer/>

механизмов, позволяющий трансформерам превосходить по качеству другие модели. Суть механизма самовнимания заключается в том, чтобы не только учитывать весь контекст, но и то, насколько каждый элемент последовательности важен для текущего или влияет на него. На этапе слоя внимания проводятся следующие вычисления:

1. Эмбединг на входе перемножается с тремя матрицами весов: query, key и value. Матрицы весов могут иметь размерность, отличающуюся от размерности эмбединга - это позволяет сократить вычисления.

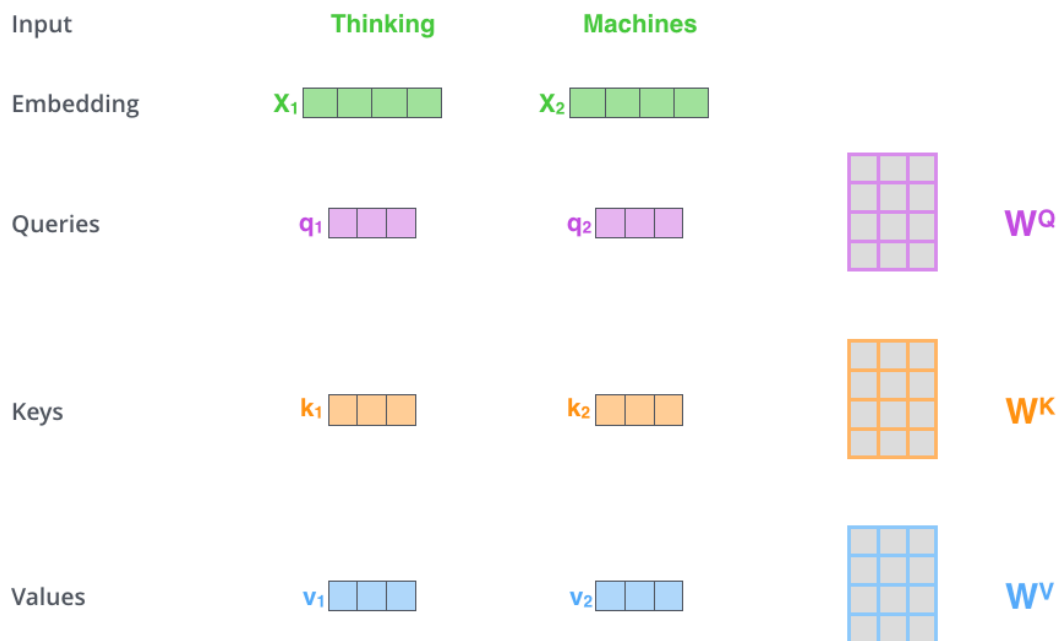


Рисунок 4. Векторы key, query и value

Источник: Jay Allamar

2. Эмбединг-query данного токена перемножается с соответствующими эмбедингами value для каждого токена, которые подавались в энкодер.

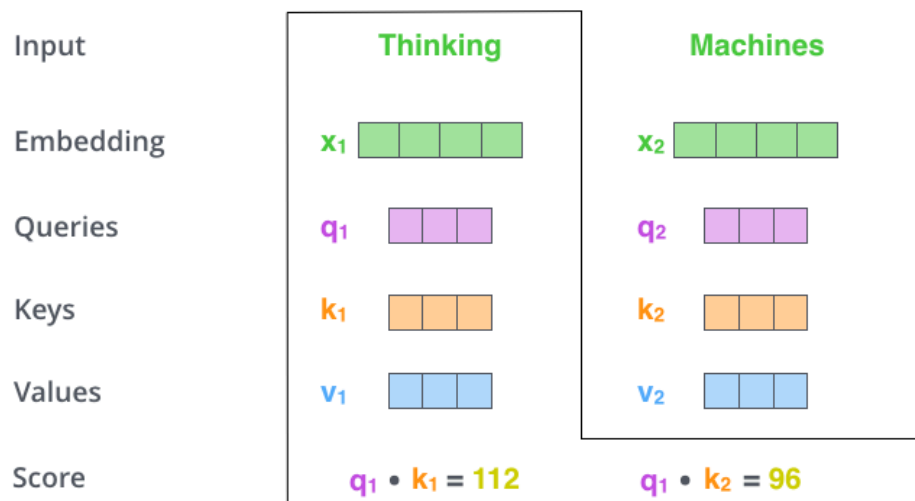


Рисунок 5. Произведение векторов query и key

Источник: Jay Allamar

3. Полученное произведение делится на квадратный корень из размерности key-вектора

4. Результат проходит через softmax-функцию, которая возвращает числа от 0 до 1 - данное число можно интерпретировать как важность каждого из слов контекста для данного слова.

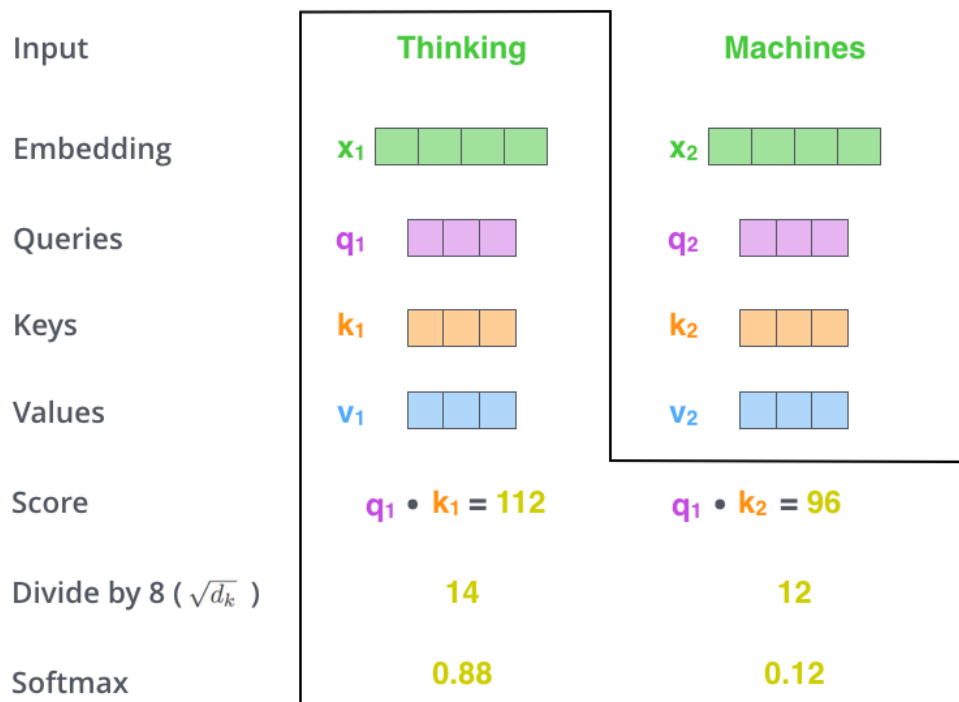


Рисунок 6. Получение softmax-score для каждого токена

Источник: Jay Allamar

d_k – размерность k-вектора,

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}, \text{ где}$$

$z = [z_1, z_2, z_3, \dots, z_K]$ – входной вектор,

z_i – i – ый элемент входного вектора,

K – общее число классов.

5. Полученное значение умножается на векторы value соответствующих токенов, затем считается взвешенная сумма для всех произведений value на результаты softmax. Итоговый вектор – выход из слоя внимания для одного токена.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Так выглядит одна “голова” самовнимания. Однако, для того, чтобы расширить представление модели о взаимосвязи слов, в каждом энкодере не один, а несколько таких механизмов, каждый с собственной матрицей весов. Результирующие векторы всех голов конкатенируются и перемножаются с матрицей весов для уменьшения размерности. Полученный вектор проходит через нормализацию и полносвязный слой энкодера, а после передается в следующий энкодер. Эти вычисления проводятся параллельно для всех элементов последовательности.

Выше было описано то, что происходит внутри энкодера, но нужно обратить внимание на еще один этап вычислений: прежде чем попасть в энкодер, эмбединги токенов перемножаются с эмбедингами позиции токена - его линейного расположения в последовательности. Эта операция позволяет учитывать расстояние между токенами в тексте.

Блок декодеров работает схожим с блоком энкодеров образом, однако имеет свои особенности. На все декодеры блока декодеров подаются векторы key и value из последнего энкодера. Помимо механизма самовнимания, его линейного слоя и нормализации, в декодере есть еще один элемент - механизм кросс-внимания (cross-attention) или энкодер-декодер внимания (encoder-decoder attention) со своим полносвязным слоем и нормализацией. Он располагается после механизма самовнимания с его линейным слоем и перед полносвязным слоем. Его задача - определить, какие векторы из переданных энкодером должны больше повлиять на текущий токен, а какие меньше. В качестве query в механизм кросс-внимания попадает выход из слоя самовнимания, а векторы key и value из энкодера. С ними производятся те же самые вычисления, что и в слое самовнимания, а результат проходит через линейный слой, нормализацию и попадает в полносвязный конечный слой.

Еще одна важная деталь, отличающая декодер от энкодера - это использование масок. Во время тренировки декодер генерирует все элементы последовательности параллельно, как и в энкодере, но при этом нельзя, чтобы он “видел” следующие за ним слова. Для этого все следующие токены маскируются - заменяются нулями. На инференсе - работе модели после тренировки - токены генерируются не последовательно, а авторегрессивно, то есть сгенерированный токен подается на вход декодера во время генерации следующего токена. Так происходит, пока декодер не вернет специальный токен, обозначающий конец предложения.

BERT

BERT - крупная языковая нейронная модель, предназначенная для анализа языковых данных [Devlin J. et al., 2019]. Изначально BERT, а именно BERT-base и BERT-large - это названия конкретных моделей, которые стали state-of-art в мире NLP, однако сейчас существует множество конфигураций этой модели, такие как RoBERTa, DistilBERT. Все эти модели называются моделями класса BERT. Отличительными чертами данного класса моделей, которые в комплексе обеспечивают высокие показатели на downstream tasks, являются двунаправленность и использование масок. Кроме того, BERT состоит только из энкодера, что соответствует задачам, которые он выполняет - анализ текста (в противопоставление генерации). Задача BERT извлечь необходимые признаки из текстовых данных, для этого достаточно энкодера.

BERT - двунаправленная модель, то есть она параллельно обрабатывает все токены. Двунаправленность позволяет учитывать и левый, и правый контексты токенов, но в то же время влечет проблему: токен использует собственные query, key и value для создания собственного же эмбединга, а это влечет то, что модель просто будет использовать вход для представления его же. Чтобы избежать этого, часть токенов маскируется. В стандартной конфигурации BERT маскинг применяется к 15% всех токенов, выбранных случайным образом. При этом 80% из них заменяются на токен [MASK], 10% заменяются на случайно выбранный токен и 10% не изменяются. Задача модели - предсказать токен по контексту - стандартная задача языкового моделирования.

Другая задача, на которой производился pre-training модели - задача предсказания следующего предложения. Для тренировки используются пары предложений, 50% которых - предложения, которые следуют в тексте друг за другом, а 50% - случайно взятые предложения. Задача модели - предсказать следуют предложения друг за другом или нет. Во время обучения в начале каждого предложения ставится специальный [CLS] токен, он будет использоваться как представление всего предложения. Это возможно потому что выход из механизма самовнимания для самого токена - это вектор, учитывающий все токены предложения. Кроме того, эмбединги токенов и позиционные эмбединги суммируются с сегментными эмбедингами (segment embeddings), которые маркируют, к какому предложению относится токен. Так как для нашей работы мы выбрали задачи классификации текстов, а не токенов, нам необходимо, чтобы тексты были представлены, как один вектор. Для этого мы будем использовать [CLS]-токен.

Fine-tuning

Fine-tuning - это метод, при котором предобученные языковые модели используются для большого спектра задач: классификация токенов и текстов, определение тональности, суммаризация, и многие другие. Предобученные модели, одной из которых является BERT, обучаются на большом объеме языковых данных для semi-supervised задач языкового моделирования. После тренировки предобученные модели имеют набор эмбедингов, которые можно использовать для вторичных задач. Преимущество этого подхода состоит в том, что для тренировки не требуется тренировать модель “с нуля”, что требует большого количества тренировочных данных, времени и вычислительных ресурсов. Вместо этого используются уже предобученные эмбединги, а обучается лишь полносвязный слой, который и выполняет поставленную задачу. Для fine-tuning, в отличие от pre-training, требуются размеченные данные.

Форматы разметки

Для обогащения предобученных эмбедингов BERT мы используем формат семантическую часть формата трехуровневой разметки CoBaLD [Petrova et al. 2023]. Рассмотрим для начала другие форматы семантической разметки, ранее использовавшиеся для сходных задач, их достоинства и недостатки.

FrameNet

FrameNet – это проект, цель которого – создать корпус с разметкой фреймов, а также исчерпывающее описание всех фреймов и их взаимодействия [Baker C. F. et al. 1998]. Фреймы – это термин, введенный в теории Frame Semantics, предложенной лингвистом Чарльзом Дж. Филлмором. “Фреймы отражают социальные и культурные контексты, а также типичные сценарии, в которых слова используются” [Тищенко А. А. 2024]. Согласно [Baker C. F. et al. 1998], фреймы во FrameNet объединены в иерархию: от общих фреймов ситуаций до более конкретных, например, к домену MOTION относится фрейм TRANSPORTATION. Описание фреймов снабжается их аргументной структурой, схожей со структурой глубинных падежей и тета-ролей, но в отличие от последних, аргументная структура во FrameNet индивидуальна для каждого фрейма, а сами роли (аргументы) называются элементами фрейма (Frame Elements или FE). Так, в список ролей для фрейма MOTION входят MOVERS, MEANS of transportations и PATHS. Субфреймы (фреймы, находящиеся ниже в иерархии) имеют ту же аргументную структуру, что и домен, но в нее также могут входить новые специфичные

для данного сабфрейма роли, которых нет в домене. Во FrameNet также представлены группы элементов фрейма (Frame Elements Groups) - комбинации элементов фрейма, которым соответствуют конкретные актантные структуры. Все возможные комбинации групп элементов фрейма снабжаются примерами из корпуса, причем приводятся все возможные синтаксические актантные структуры, которые могут соответствовать данной группе элементов фрейма.

Prague Textogrammatical Graph (PTG)

PGT - это формат разметки, основанный на теории Functional Generative Description of natural language [Sgall et al., 1986]. Он используется в параллельном корпусе текстов на английском и чешском языке Prague Dependency Treebank [Hajic J. et al. 2001]. Разметка корпуса имеет три уровня: морфологический, аналитический или уровень поверхностный синтаксис (surface syntax) и текстограмматический уровень или глубинный синтаксис (underlying syntax). На морфологическом уровне размечаются токены, леммы и морфосинтаксические категории. Аналитический уровень представляет собой дерево зависимостей и аналитические функции (субъект, объект, предикат и проч.). [Hajic J. et al. 2001]

Основной элемент текстограмматического уровня - дерево зависимостей, узлами которого являются “автосемантические слова” - слова, имеющие лексическое значение. Слова, имеющие только грамматическое значение (например, вспомогательные глаголы) не выделяются. Зависимости между узлами (юнитами) называются фанкторами. В разметке используются 72 фанктора. Они поделены на ряд категорий: аргументы (агенс, пациенс, адресат и проч.), модификаторы (время, место, образ действия), и другие “технические категории”: конъюнкция и противопоставление, отрицание и т. д. Юнитам присваиваются фанкторы в соответствии с их валентным фреймом - обязательными и факультативными валентностями. Данный формат разметки подразумевает восстановление эллипсиса: если валентность юнита должна быть заполнена либо словом, либо специальной категорией. Кроме того, для юнитов указываются некоторые грамматические категории, например, число у существительных, род, модальность и время у глаголов. К текстограмматическому уровню также относится и разметка топика и фокуса.

UCCA

UCCA - это формат многослойной семантической разметки, основанный Basic Linguistic Theory (BLT) [Dixon, 2005; 2010a; 2010b; 2012]. Данный формат создавался как универсальный, не привязанный к конкретному языку, а также доступный в использовании не-лингвистами [Abend O., Rappoport A. 2013]. На данный момент существуют корпуса на английском, немецком и французском языках, размеченные в формате UCCA.

Разметка в формате UCCA представляет собой ориентированный граф в узлах которого находятся юниты - базовые единицы разметки. Листья называются терминалами. Юниты могут быть как терминалами, так и объединением элементов. В вершине графа находится Сцена (Scene). Сцена – это событие или состояние, как правило, имеющие временные и пространственные характеристики. Сцены делятся на два вида: процесс (Process) и состояние (State). Сцена включает в себя участников (Participants) и сирконстантные отношения (Adverbials).

Юнитами второго слоя, в который не входит сцена, являются Центр (Center), Elaborators, Connectors и Relators. Центр - это основная единица юнита не-сцены, без него юнит не может быть образован. Elaborators - отношение, применимое к одному центру. Connectors - отношение между двумя и более однородными юнитами, а Relators – между разнородными. Те юниты, которые не связаны с другими никаким отношением, “пустые” семантически, размечаются как функции (Functions). Помимо лексически выраженных в предложении юнитов, в UCCA выражаются нулевые актанты, называемые Implicit Units.

AMR

Задача AMR - создать семантическую разметку, не зависящую от синтаксической структуры предложения и хорошо применимую для английского языка [Banarescu L. et al. 2013].

Разметка AMR представляет собой направленный граф с labeled узлами и листьями. Узлы графов создаются с опорой на неодавидсоновскую семантику [Davidson, 1969]. Узлами графов могут быть сущности, события, качества и состояния. Узлы графов соединяются отношениями, в AMR их порядка 100. Размечаются следующие типы отношений:

- ключевые роли - аргументы фрейма;
- основные семантические отношения (например, бенефактив, причина, сравнение, условие и т. д.);
- отношения для величин;
- временные отношения;
- отношения для списков.

В формате AMR размечается отрицание, инверсия, модальность, кореференция и копулы. Для вопросов с вопросительным словом используется специальный тэг amr-unknown для обозначения аргумента, на месте которого состоит вопросительное слово. Предлоги опускаются, поскольку их функцию в разметке выполняют типы отношений. Местоимения в разметке не используются, вместо них используется отсылка к референту. Если референт лексически не выражен, используется местоимение he.

СемЭТАП

СемЭТАП - это модель семантической разметки, которая используется в корпусе русских текстов SemOntoCor [Boguslavsky I. M. et al. 2023]. Разметка основана на теории И. Мельчука Текст-значение [Mel'čuk I. 2012, 2013, 2015]. В разметке разделяется два вида структур - Базовая семантическая структура, которая представляет собой семантику предложения, и Расширенная семантическая структура, которая также учитывает экстралингвистические знания. Сама разметка может быть представлена в виде направленного графа, где узлы - это элементы семантической онтологии - ОнтоЭТАП, а ребра - семантические отношения.

Разметка, с одной стороны, стремится отразить семантику предложения вне семантической структуры, с другой стороны, сохранить важные с точки зрения дискурса элементы, например, модальность. Семантика слов раскладывается на

концепты, иерархия которых представлена в онтологии. Семантические отношения представляют собой свойства концептов, перечисленные в определении концепта в онтологии.

Среди рассмотренных нами форматов разметки есть те, которые целенаправленно не учитывают синтаксическую структуру, чтобы семантический образ предложения не зависел от синтаксиса. В частности, предложения, имеющие разную синтаксическую структуру, но одинаковую семантику, в таких форматах разметки должны выглядеть одинаково. К таким корпусам можно отнести FrameNet, UCCA и AMR. В то же время, не все корпуса учитывают лексические значения слов. Так, в PGT, UCCA и AMR разметка производится по семантическим ролям, но не по лексическим значениям слов.

CoBaLD

В нашей работе мы используем формат разметки CoBaLD (Compreno Based Linguistic Data, предложенный в 2023 году) [Petrova et al. 2023]. Разметка в данном формате имеет три уровня: морфологический, синтаксический и семантический.

Цель проекта CoBaLD - создать универсальную разметку трех основных уровней языка, имеющую простую и понятную структуру, а также создать датасет русских текстов, размеченный в данном формате. Кроме того, в рамках проекта был создан нейронный парсер для автоматической разметки в формате CoBaLD [Баяк И.С 2024].

В качестве формата для разметки морфологии и синтаксиса был выбран формат Universal Dependencies (UD) [De Marneffe et al. 2006], как широко используемый и универсальный для разных языков. UD имеет следующие основные принципы:

- основной единицей считается токен;
- всем токенам, кроме предиката, приписывается вершина - другой токен.
Вершиной предложения считается предикат;
- разметка включает в себя часть речи, морфологические свойства слов и тип грамматических отношений между словами.

UD не включает семантическую разметку, поэтому в качестве формата семантической разметки была выбрана модель Compreno [Petrova M. A., 2014]. В Compreno разметка производится по двум параметрам: семантический класс и семантическая роль.

Семантический класс (СК) – семантическое поле, в котором лежит значение слова.

Семантическая роль (СР) – глубинная позиция.

Набор семантических классов в Compreno представляет собой иерархию семантических областей, в которых находится значение слова. Семантические роли отражают связи между словами, например, адьюнкты, характеристики, спецификаторы и другие. В оригинальном формате Compreno представлено более 200000 семантических классов и более 330 семантических ролей. Это количество кажется избыточным и затруднит обучение модели на ограниченном объеме размеченных данных. По этим причинам иерархия семантических классов была сокращена: вместо терминальных единиц иерархии использовались их гиперонимы. В списке семантических ролей дробные типы отношений были заменены более общими. Так, например, типы Инструментов были объединены под одним тегом Инструмент:

Instrument		to write [with a pen]
Instrument_Being	Instrument	to attack [with remaining army]
Instrument_Cognitive		to understand [with one's heart]
Instrument_Time		to be punished [by 30 years in prison]

Рисунок 7. Замена подклассов класса *Instrument* на их гипероним
[Petrova et al. 2023, Figure 1]

Формат *Compreno* имеет свою разметку синтаксической структуры, которая частично не совпадает с разметкой *UD*. В этом случае разметку синтаксических вершин требовалось вручную менять, чтобы устранить несоответствия в разметке.

Задачи

В нашей работе мы выбрали задачи классификации текстов, а именно анализ тональности и детекцию токсичности. Выбор задач классификации текстов обусловлен тем, что для этого типа задач, в отличие от задач классификации токенов, экспериментов по обогащению эмбедингов семантической информацией в формате *CoBaLD* не проводилось.

Анализ тональности

Анализ тональности - классическая задача в NLP. Ее решением занимались еще в 2000-ых, и в наше время она не утратила своей актуальности. Анализ тональности включает в себя ряд подзадач: классификация мнений (*Subjectivity classification*), классификация тональности (*Sentiment classification*), детекция спама мнений (*Opinion Spam Detection*)³ и выявление имплицитного языка (*Implicit Language Detection*): детекция сарказма, иронии, юмора. Решения этих задач широко применяются на практике: для бизнеса и некоммерческих организаций, например, здравоохранения, важно знать мнение потребителей товаров или услуг, а также общее настроение рынка для эффективной работы. [Wankhade M., et al., 2022, Ribeiro F. N. et al., 2016]

Анализ тональности может производиться на следующих уровнях: на уровне текста, предложения и фразы. На уровне текста анализ сентимента производится не часто, особенно, если текст большой и разные его части содержат разные по тональности элементы. Уровень предложения можно назвать базовым уровнем для анализа тональности, особенно если речь идет о коротких отзывах, комментариях и проч. Анализ тональности для фразы не получили широкого распространения и используется для решения специфических задач.

В нашей работе анализ будет проводиться формально на уровне текста, однако объем текстов как правило небольшой, часто одно или несколько предложений. В виду этого проблемы, которые связаны с анализом больших по объему текстов, для нашей работы не релевантны.

Для анализа тональности применяются те же методы, что и для других задач NLP - различные механизмы машинного обучения и нейронные сети. Необходимо также обратить внимание на наиболее простые методы, которые сами по себе дают не дают

³ Opinion spams, often known as fraudulent or phone reviews, are well-written comments supporting or criticizing a product for their benefit. [Wankhade M., et al., 2022]

хорошего результата, однако могут помочь улучшить качество более продвинутых методов.

Наиболее примитивный подход - это подход, основанный на лексиконе. Создается словарь токенов, где каждому токenu приписывается оценка от 1 до -1, где 1 - это положительная тональность, а -1 - отрицательная. Тональность текста (предложения, фразы) определяется суммой оценок токенов в тексте. Такой подход очень прост вычислительно и не требует тренировочных данных, однако, очевидно, он не учитывает контекстные значения токенов. На практике большая часть слов приобретает негативную или позитивную окраску в зависимости от контекста, поэтому данный метод как самостоятельный не используется.

Корпусной подход

В данном подходе используются размеченные для конкретной задачи анализа тональности корпуса. Помимо очевидных применений корпусов для обучения языковых моделей, если два более простых метода.

Статистический метод

Данный метод учитывает частотность появления конкретного токена в разных контекстах. В зависимости от того, в каких текстах токен встречался чаще в корпусе - в размеченных как положительные или отрицательные, определяется его тональность.

Семантический метод

Подход основывается на материалах типа WordNet, где легко выделить тонально окрашенные слова, а также их синонимы, антонимы и гиперонимы. Тональность текста основывается на сумме оценок определенных заранее слов, отражающих тональность, и связанных с ними слов в тезаурусе.

Словарный подход

Изначально составляется небольшой словарь из слов, которым приписывается негативная или позитивная тональность. Затем этот словарь дополняется синонимами и антонимами этих слов из ресурсов типа тезаурусов или WordNet. Предполагается, что синонимы будут иметь одинаковую тональность, а антонимы - противоположную. Затем этот словарь дополняется в процессе анализа текстов новыми, определяемыми по контексту, синонимами и антонимами.

Хотя качество решений анализа тональности за последние 20 лет сильно возросло, остается ряд проблем, требующих решения. Большая часть этих проблем актуальна для NLP в целом.

1. Отрицание и антонимы. В современных нейронных моделях и методах машинного обучения эмбединги слов строятся, так или иначе основываясь на контексте слов. Близкими в векторном пространстве оказываются те слова, которые встречаются в схожих контекстах. Для большинства случаев так и есть, но это не работает в случае с антонимией и отрицаниями. Антонимы могут встречаться в очень похожих контекстах, но при этом их семантика противоположна. То же самое происходит и с отрицанием: два высказывания могут отличаться только наличием частицы “не” и иметь противоположные значения.
2. Обработка иронии и сарказма. Суть этих явлений такова, что слова, которые имеют положительную тональность, используются для выражения негативного

отношения и наоборот. Это представляет собой проблему для алгоритмов машинного обучения и нейронных моделей, поскольку не существует эксплицитных маркеров, которые используются для выражения сарказма.

3. Неформальный стиль общения. В неформальном общении, помимо черт, общих для всех стилей, могут использоваться не общепринятые акронимы и сокращения, эмодзи и смайлы. Во-первых, неформальный стиль речи очень изменчив, и в обучающих данных, которые собраны год назад, может не быть тех явлений, которые используются в разговорной речи в настоящий момент. Во-вторых, если модель обучалась на текстах, написанных в формальном стиле, например, газеты, новости, то она не будет “знать” те явления, которые используются лишь в неформальном общении.
4. Грамматические ошибки. Грамматические ошибки – еще одна проблема, особенно актуальная для данных из соцсетей, форумов и других неофициальных ресурсов. В неформальной переписке пользователи часто совершают ошибки, и языковой модели нужно научиться определять слово, которое на самом деле хотел написать автор.
5. Снятие омонимии. Омонимы - слова, имеющие одинаковую звуко-буквенную форму, но имеющие не связанные между собой значения. Для анализа тональности, как и для многих других задач NLP языковая модель должна определять, какое из слов-омонимов перед ней.
6. Доступность обучающих данных. Для обучения языковых моделей анализу тональности, требуется большое количество обучающих данных, причем часто эти данные должны быть размечены. Во-первых, для разметки требуются человеческие ресурсы, которые не всегда доступны, во-вторых, большая часть существующих датасетов состоит из текстов на английском языке, а вот датасетов для языков, не так широко распространенных, значительно меньше, или может вообще не быть.
7. Вычислительные ресурсы. Во время обучений нейронных моделей производится огромное количество вычислений, для которых требуется мощная видеокарта.

Детекция токсичности

Детекция токсичности - сравнительно новая задача в NLP. Одни из первых крупных исследований на эту тему - соревнования на платформе Kaggle, которые проводили Jigsaw/Conversation AI [Jigsaw 2018, 2019, 2020]. В качестве обучающих данных был представлен крупный датасет с разметкой токсичности для ряда языков, причем для некоторых языков есть деление на более дробные подтипы: сильная токсичность, атака идентичности, оскорбление, ругательства, угроза. Определения каждого из этих типов даны на сайте проекта⁴.

На данный момент нет единого общепринятого определения токсичности. Определение, которое используют многие исследователи токсичности принадлежит Jigsaw: “грубое, неуважительное или беспричинное высказывание, которое может заставить человека покинуть дискуссию”. Другой подход используют авторы [Dementieva D. et al., 2022]: вместо того, чтобы давать определение токсичности эксплицитно, они дали разметчикам примеры токсичных (по мнению авторов статьи) высказываний.

⁴ https://developers.perspectiveapi.com/s/about-the-api-attributes-and-languages?language=en_US

Задача выявления токсичности стоит наряду с такими задачами как детекция агрессии, буллинга, hate speech и обценной лексики. Все эти задачи стали активно обсуждаться в последние несколько лет, в связи с практической необходимостью: многие социальные сети и другие медиа ресурсы стали искать способы автоматически выявлять токсичную речь и другие упомянутые выше явления и устранять их тем или иным способом. В частности, появилась задача детоксификации - преобразования токсичных высказываний в не-токсичные [Dementieva D. et al., 2022].

В рассмотренных нами источниках, авторы работ по детекции токсичности использовали различные типы нейронных языковых моделей CNN [Georgakopoulos S. V. et al., 2018], LSTM [Smetanin S., 2020], BERT [Dementieva D. et al., 2022, Zampieri M. et al., 2019] В частности существуют работы по детекции токсичности и датасеты для этой задачи на материалах русского языка. Так, на соревновании RUSSE-2022 [Dementieva D. et al., 2022] был представлен первый датасет параллельных текстов для детоксификации, а в работе [Smetanin S., 2020] описывается обучение моделей класса BERT с помощью датасета, анонимно опубликованного на платформе Kagge⁵. Мы также будем использовать данный датасет.

Проблемы, которые возникают при решении задачи детекции токсичности, актуальны и для анализа тональности: доступность размеченных данных, разрешение омонимии, работа с иронией и сарказмом.

Обогащение эмбедингов семантикой

Попытки добавить эксплицитную семантическую информацию в эмбединги токенов для решения разных задач уже проводились. Рассмотрим методологию некоторых из них.

В работе [Kim J. K. et al., 2016] авторы обращают внимание на проблему, упомянутую нами выше: неспособность языковых моделей верно обрабатывать антонимию. Причиной этого является то, что из-за схожего контекста антонимы оказываются близко в векторном пространстве. Чтобы решить эту проблему, авторы используют WordNet, Paraphrase Database (PPDB) и словарь Макмиллана.

WordNet – это тезаурус, в котором слова объединены по “когнитивной близости”.

Отдельные концепты, которые во многом основываются на лексической близости слов называются синсеты (synsets). [Fellbaum C., 1998]

Paraphrase Database (PPDB) – это датасет (база данных), в котором собраны перифразы на английском и испанском языках. Для каждой пары перифраз даются оценки, определяющие степень их семантической близости. [Ganitkevitch J. et al., 2013]

Словарь Макмиллана (Macmillan Dictionary) – словарь, предназначенный для изучения английского языка. Помимо определений слов, в нем приводятся частотные коллокации. [Rundell M., 2002]

Авторы статьи предлагают следующую идею: нужно уменьшить косинусное расстояние между эмбедингами синонимов и увеличить косинусное расстояние между эмбедингами антонимов.

⁵ <https://www.kaggle.com/datasets/blackmoon/russian-language-toxic-comments>

Для этого берутся пары синонимов антонимов, а также связанные (related) слова, которые при этом не являются синонимами (например, commute и ride). Для сближения или отдаления эмбеддингов используются функции, приведенные ниже.

1) Для сближения векторов синонимов

$$SC(V) = \sum_{(u,w) \in S} \tau(\delta - \cos(u_w, w_u)), \text{ где}$$

S — набор пар антонимов,

$\delta = 1$.

2) Для сближения related words

$$KN(V, V^0) = \sum_{i=1}^N \sum_{j \in N(i)} \tau(\cos(u_i, w_j) - \cos(u_i^0, w_j^0)), \text{ где}$$

V^0 — исходная матрица словаря,

N — размер словаря,

$N(i)$ — изначальное количество соседних слов для i — ого слова.

3) Для отдаления антонимов

$$AF(V) = \sum_{(u,w) \in A} \tau(\cos(u_w, w_u)), \text{ где}$$

$\tau = \max(0, x)$,

V — матрица словаря,

A — набор пар антонимов,

u_i — i — тый ряд в V .

В качестве исходных эмбеддингов слов авторы использовали эмбеддинги GloVe. После обогащения семантикой эти эмбеддинги были использованы для решения задачи определения намерений при помощи модели LSTM. Данная методика позволила улучшить результаты решения задачи по сравнению с бэйзлайном.

Стоит заметить, что хотя данный подход работает, он лишь помогает решить проблему антонимов в NLP, однако не вносит собственно семантической информации в эмбеддинги. К тому же, в модели используются статические эмбеддинги, которые показывают результаты хуже, чем динамические.

Авторы еще одного подхода к обогащению эмбеддингов токенов семантической информацией [Pittaras N. et al., 2021] тоже используют WordNet, но предлагают следующую методологию.

1. В качестве эмбеддингов токенов берутся эмбеддинги word2vec, обученные на используемых в исследовании текстах.
2. Получение векторов семантики проходит в несколько этапов
 - a. Из WordNet берется синсет для искомого токена. Нужно заметить, что каждый синсет может содержать несколько значений слов (если таковые имеются). Каждое значение маркировано соответствующим индексом, а также для каждого значения указывается часть речи.
 - b. Для определения того, какое конкретно значение имеет токен в тексте, используется три подхода. Первый, самый простой - берется наиболее

частотное значение. Второй подход учитывает часть речи. Таким образом, фильтруются значения в синсете, часть речи которых отличается от части речи токена в тексте. Третий подход подразумевает использование контекстов. Для каждого значения слова строятся эмбединги на основе контекстов из примеров предложений для каждого значения в синсете и из определения для элемента синсета. Далее определяется косинусное расстояние с эмбедингом слова в тексте и выбирается самый близкий.

- c. WordNet позволяет учитывать не только синонимы слов, но и гиперонимы. Для этого авторы статьи берут n гиперонимов для каждого токена (в итоге авторы остановились на $n=3$), и каждому гиперониму присваивают вес, в зависимости от удаленности от искомого синсета. Веса начинаются с 0.6 и уменьшаются в геометрической прогрессии.
 - d. Построение векторов семантики для каждого текста происходит следующим образом. Берется многомерное пространство, где каждое измерение - один синсет, а тексты располагаются в нем по частотности каждого синсета. Таким образом получается вектор для каждого текста.
 - e. К каждому вектору текстов применяется процедура взвешивания, аналогичная TF-IDF: значение частотных для всех документов концептов уменьшается.
3. Готовые векторы семантики конкатенировались с эмбедингами слов, после чего были взяты за исходные в модели DNN. В качестве задач авторы взяли выявление тем на двух датасетах.

Данный метод, как и предыдущий, позволил улучшить качество модели, и уже учитывает непосредственно лексическое значение слова. Однако, следует заметить, что дизамбигуация значения слова при помощи лишь части речи или частотности - очень грубое решение, а примеров для построения контекстных векторов в примерах и определениях в WordNet может быть недостаточно.

Авторы третьего подхода [Zhang Z. et al., 2020], который мы рассмотрим, использовали для обогащения эмбедингов семантикой не лексические значения слов, а семантические роли. Для этих целей авторы выбрали формат разметки PropBank, аналогичный AMR, рассмотренному выше. Для разметки семантических ролей токенов используется предобученная модель, что отличает эту работу от двух предыдущих. Для каждого токена получается два варианта разметки, где один - менее детализированная версия другого. Для каждого токена в последовательности формируется вектор из n возможных семантических ролей, затем векторы передаются в рекуррентную нейронную сеть BiGRU, которая возвращает эмбединги для семантических ролей.

Так как BERT возвращает эмбединги для подслов, а семантические векторы были получены для исходных токенов = слов, необходимо привести размерность матрицы эмбедингов к размерности матрицы векторов семантики. Для этого авторы статьи использовали CNN. Полученные эмбединги токенов и векторы семантики были сконкатенированы, и использовались для fine-tuning'a для различных задач классификаций текстов. Как и другие варианты обогащения семантикой, этот показал результаты, превосходящие бейзлайн.

Данный вариант обогащения семантикой кажется нам наиболее прогрессивным, поскольку использует предобученную модель для семантической разметки, а так же

исходные эмбединги модели BERT - наилучшей на данный момент для анализа текстов.

Мы учли плюсы и минусы различных вариантов обогащения эмбедингов токенов семантикой и выбрали следующую конфигурацию:

- для обогащения был выбран формат разметки CoBaLD, так как он представляет легко имплементируемую разметку, которая при этом учитывает и семантические роли, и лексические классы;
- используем предобученный парсер для разметки датасета в этом формате;
- построим эмбединги семантики при помощи bi-LSTM;
- сконкатенируем их с предобученными эмбедингами BERT и используем их для fine-tuning для выбранных нами задач классификации текстов.

Нужно заметить, что уже существуют работы, посвященные обогащению эмбедингов слов на базе разметки в формате CoBaLD. В работе [Гусева И. М., 2024] для обогащения использовались статические эмбединги Word2Vec, а для оценки модели были взяты несколько downstream-задач анализа текстов. По результатам исследования, модель с обогащенными эмбедингами показывает лучшие по сравнению с бэйзлайном результаты на задачах определения тональности и выявлению текстов, сгенерированных ИИ, по отношению к именованным сущностям. А вот для распознавания позиции разницы с бэйзлайном не было.

В работе [Тищенко А. А., 2024] также используется формат разметки CoBaLD, но, в отличие от предыдущей работы, на задачах классификации токенов. Кроме того, автор данной работы использует предобученные эмбединги BERT, которые являются динамическими. В данном случае обогащение эмбедингов BERT семантикой помогло улучшить качество модели на задаче выявления именованных сущностей, хотя и незначительно, но не повлияло на качество решения задачи POS-tagging.

Практическая часть

Датасеты

Как было сказано выше, для проверки нашей гипотезы мы выбрали две задачи классификации текстов. Одна из них – анализ тональности – классическая задача в NLP, другая – детекция токсичности – новая задача, которая активно исследуется в последние несколько лет.

Язык, выбранный нами для исследования – русский, соответственно, для обучения модели нам нужны были размеченные датасеты для обеих задач.

Для задачи анализа тональности мы выбрали датасет, размещенный на платформе huggingface, который содержит тексты комментариев с сайта Кинопоиск⁶.

Характеристики датасета следующие:

- Объем:
 - Тренировочная часть – 10500 текстов;
 - Валидационная часть – 1500 текстов;

⁶ <https://huggingface.co/datasets/ai-forever/kinopoisk-sentiment-classification>

- Тестовая часть – 1500 текстов.
- Разметка тональности:
 - 0 – Bad - негативная тональность;
 - 1 – Neutral – нейтральная тональность;
 - 2 – Good – позитивная тональность.
- Баланс классов: $\frac{1}{3}$

Мы выбрали данный датасет, поскольку он содержит подходящую для нас разметку тональности, имеет достаточный объем и идеальный баланс классов, что очень важно для тренировки модели.

Для задачи детекции токсичности был выбран датасет с платформы Kaggle⁷. Нужно заметить, что готовых датасетов на русском языке очень мало, очевидно, в виду того, что задачей детекции токсичности стали заниматься недавно. Выбранный нами датасет имеет наилучший баланс классов из всех рассмотренных вариантов и разметку, подходящую под наши задачи. Датасет состоит из комментариев с сайтов 2ch⁸ и Pikabu⁹. Объем датасета 14412, который мы затем поделили на три части:

- Тренировочная часть – 11 529 текстов (8/10 датасета)
- Валидационная часть – 1 441 текст (1/10 датасета)
- Тестовая часть – 1 441 текст (1/10 датасета)

Разметка датасета бинарная – 1 – токсичный комментарий, 0 – нетоксичный. Баланс классов: $\frac{1}{3}$ с меткой 1, $\frac{2}{3}$ с меткой - 0.

Данный датасет был рассмотрен также в работе [Smetanin S., 2020]. Авторы статьи провели ручную разметку части данного датасета для подтверждения валидности разметки. Эксперимент показал высокий уровень согласия аннотаторов – 0.81 (коэффициент Альфа Кримпендорфа), что подтверждает качество разметки датасета.

Метрики в машинном обучении

Для оценки результатов обеих моделей мы использовали стандартные метрики, используемые в машинном обучении.

Кросс-энтропия

Кросс-энтропия – это классическая функция ошибки, применимая при классификации, где ожидаемые и реальные значения дискретны [Goodfellow I. et al., 2016]. Для многоклассовой классификации формула кросс-энтропии выглядит следующим образом:

$$CrossEntropyLoss(x, target) = - \sum_i^N (target_i \cdot \log(x_i)),$$

где N – количество классов, $target$ – ожидаемое значение, x – предсказание модели, i – номер пары элементов.

Кросс-энтропия принимает значения в интервале $[0, +\infty)$, где 0 – все предсказания модели верны.

Accuracy

⁷ <https://www.kaggle.com/datasets/blackmoon/russian-language-toxic-comments>

⁸ <https://2ch.hk/>

⁹ <https://pikabu.ru/>

Accuracy – точность – доля правильных ответов модели ко всем ответам.

$$Accuracy = \frac{1}{N} \sum_{i=1}^N [x_i = target_i]$$

где N – общее количество предсказаний, $target$ – ожидаемое значение, x – предсказание модели, i – номер пары элементов.

Precision – точность по положительному классу – доля правильно предсказанных объектов класса p ко всем объектам, которые модель отнесла к классу p . В случае бинарной классификации формулу precision можно записать так:

$Precision = \frac{TP}{TP+FP}$, где TP – правильно предсказанные объекты положительного класса, FP – ошибочно предсказанные объекты положительного класса

Recall – полнота – доля объектов класса p , которые предсказала модель ко всем объектам класса p . Для бинарной классификации формула выглядит так:

$Recall = \frac{TP}{TP+FN}$, где TP – правильно предсказанные объекты положительного класса, FN – неправильно предсказанные объекты положительного класса.

F1-score – метрика, агрегирующая precision и recall. F-мера представляет собой среднее гармоническое между precision и recall.

$$F1 - score = \frac{2 * precision * recall}{precision + recall}$$

Значения F-меры находятся в промежутке от 0 до 1.

Ход работы

В нашем исследовании для обеих задач мы использовали предобученную модель BertModel "DeepPavlov/rubert-base-cased" для русского языка и предобученный токенизатор BertTokenizerFast "DeepPavlov/rubert-base-cased". Для всех моделей использовались следующие конфигурации:

- Размерность эмбедингов - 512,
- Вид градиентного спуска - AdamW с параметрами по умолчанию,
- Dropout - 0.3,
- Размер батча - 32.

Обогащение семантикой проводилось следующим образом:

1. Из размеченных парсером CoBaLD [Баяк 2024] датасетов извлекались семантические роли и классы для каждого текста.
2. Полученные семантические тэги объединялись со стандартным составом батчей: токенизированные и закодированные при помощи one-hot encoding тексты, тэги для классов и attention mask.
3. Для получения эмбедингов для семантических ролей и классов для каждого мы использовали модель bi-LSTM.
4. Во время прямого проброса (forward) – этап, когда модель выдает предсказания – эмбединг [CLS]-токена, который содержит информацию о всем тексте конкатенировался (“склеивался”) с эмбедингами семантических ролей и

классов. На этом этапе и происходило собственно обогащение эмбеддингов BERT семантикой.

5. Модель делала предсказание на основе обогащенных семантической информацией эмбеддингов.

Результаты

Детекция токсичности

Параметры для обеих моделей:
 количество эпох - 3,
 learning rate - $2e-5$.

Общие результаты по тестовой выборке

Метрики	Бейзлайн	Модель, обогащенная семантической информацией
CrossEntropyLoss	0.149	0.147
Accuracy	0.947	0.944
Precision	0.952	0.949
Recall	0.940	0.936
F1-score	0.946	0.943

Таблица 1. Общие результаты по тестовой выборке для детекции токсичности.

Результаты по классам

Бейзлайн

Класс	precision	recall	f1-score
0	0.94	0.95	0.95
1	0.95	0.94	0.95
accuracy			0.95

Таблица 2. Результаты по классам для бейзлайна детекции токсичности.

Матрица предсказаний

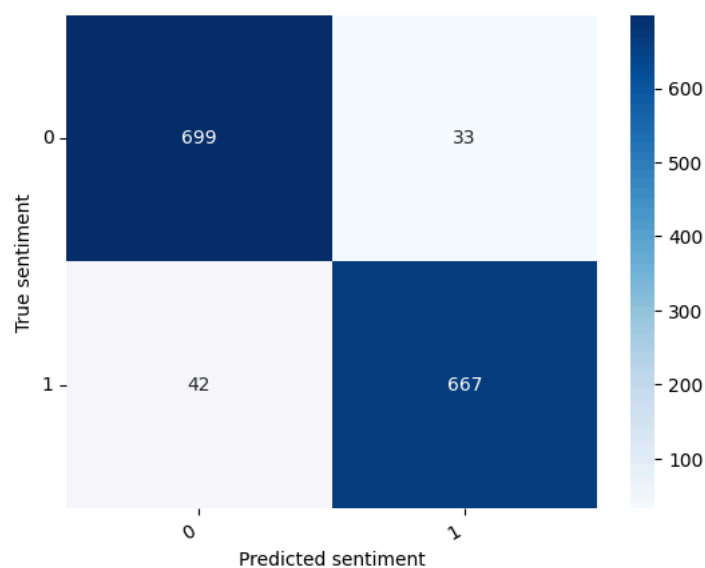


Рисунок 8. *Confusion matrix* для бейзлайна для детекции токсичности

Модель, обогащенная семантической информацией

Класс	precision	recall	f1-score
0	0.94	0.95	0.95
1	0.95	0.94	0.94
accuracy			0.94

Таблица 3. Результаты по классам обогащенной для модели детекции токсичности.



Рисунок 10. *Confusion matrix* обогащенной модели для детекции токсичности

Как можно видеть, общие метрики для бейзлайна и обогащенной семантикой модели практически не отличаются, а показатели предсказаний по классам абсолютно

идентичны. Ручная проверка ошибок, которые совершают обе модели позволила выявить следующие тенденции:

1. Большая часть неверно размеченных текстов – короткие комментарии в одно предложение.

(1) Зря тут этот пост. Теперь дом спиздят

Правильно: 0

Предсказано: 1

(2) Просто место намоленное

Правильно: 0

Предсказано: 1

2. Изначальная разметка этих комментариев кажется нам сомнительной: во-первых, часто сложно понять комментарий без широкого контекста, в котором он был написан, во-вторых, многие комментарии содержат обсценную лексику, негативные оценки и оскорбления, и, как нам кажется, должны быть размечены как токсичные.

Непонятно, почему присвоен тэг “1”:

(3) Где они учатся увечья имитировать?

Правильно: 1

Предсказано: 0

(4) Чето кажется выйдет перегиб сансары 1000 5000

Правильно: 1

Предсказано: 0

Непонятно, почему комментарий нельзя считать токсичным:

- (5) Помойку устроили в Киеве приехавшие на ЛЧ зарубежные болельщики. Они срали и ссали в центре Киева прямо на Майдане Незалежности тем самым выразили отношение к Украине А в нашей помойке они убирали за собой стадионы. Потому что у нас чисто, красиво и люди хорошие. И им совесть не позволяла оставить после себя мусор в нашей замечательной стране. Хотя им препятствовали собирать мусор но... Вот настоящее отношение к России. Россия охуенно, братан!

Правильно: 0

Предсказано: 1

3. Модель часто неверно (согласно разметке) приписывает тэг 1 (токсичный) тем комментариям, которые содержат обсценную лексику и негативно окрашенный слэнг.

(6) Вы правда считаете, что быдлокодер вашего софта на ПК и генно - кодер это один и тот же человек?

Правильно: 0

Предсказано: 1

(7) Зря тут этот пост. Теперь дом спиздят

Правильно: 0

Предсказано: 1

- (8) а смысл тратить керосин надохлых холопов когда этим же бортом можно провезти 35 тонн аргентинской муки?

Правильно: 0

Предсказано: 1

4. Многие ошибки у обеих моделей совпадают, что подтверждает тот факт, что обогащение семантической информацией не повлияло на качество модели.

Вывод

Обогащение эмбедингов модели BERT семантической информацией не повлияло на качество решения задачи детекции токсичности. Это подтверждает тот факт, что модели до и после обогащения семантикой совершают ошибки часто на одних и тех же примерах.

Анализ тональности

Гиперпараметры:

количество эпох - 5,

learning rate - 2e-5.

Общие результаты на тестовой выборке

Метрики	Бейзлайн	Модель, обогащенная семантической информацией
CrossEntropyLoss	0.918	1.012
Accuracy	0.678	0.667
Precision	0.690	0.676
Recall	0.678	0.667
F1-score	0.682	0.671

Таблица 4. Общие результаты по тестовой выборке для детекции токсичности.

Результаты по классам

Бейзлайн

Класс	precision	recall	f1-score
0	0.35	0.36	0.35
1	0.34	0.32	0.33
2	0.35	0.35	0.35
accuracy			0.35

Таблица 5. Результаты по классам бейзлайна для анализа тональности

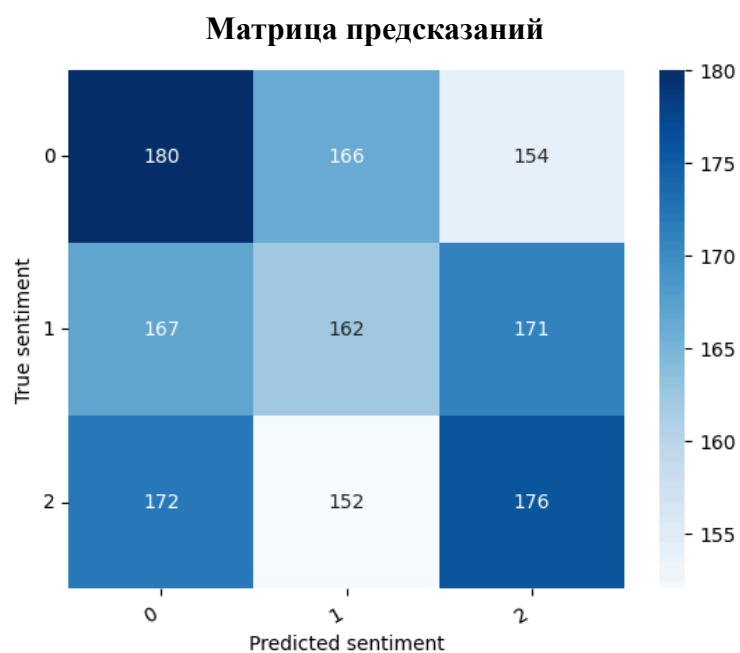


Рисунок 11. Confusion matrix бейзлайна для анализа тональности

Модель, обогащенная семантической информацией

Класс	precision	recall	f1-score
0	0.36	0.36	0.35
1	0.32	0.33	0.33
2	0.33	0.33	0.33
accuracy			0.34

Таблица 6. Результаты по классам обогащенной модели для анализа тональности

Матрица предсказаний

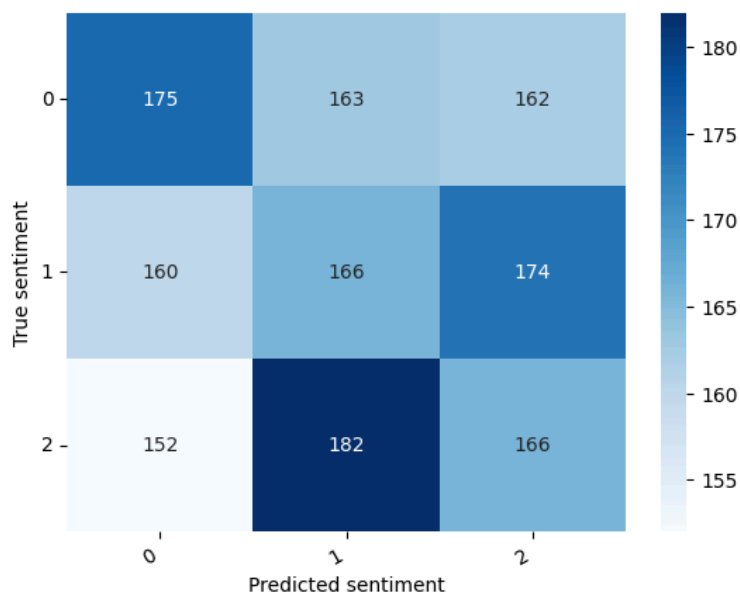


Рисунок 12. Confusion matrix обогащенной модели для анализа тональности

При работе с датасетом для анализа тональности мы столкнулись с проблемой: качество предсказаний модели на бейзлайне было очень низким по сравнению с тем, какое качество в среднем показывают модели класса BERT. После ручной проверки небольшой выборки ошибок модели на датасете стали понятны предполагаемые причины такого поведения:

1. Был взят отрывок тестового датасета размером в 5 батчей (=80 примеров), в котором модель совершила 30 ошибок. Из 30 примеров, где предсказания модели не совпадают с разметкой датасета, 10 кажутся нам размеченными неверно изначально, причем в большей части из них ошибки связаны с тэгом 1 – нейтральная оценка.

- (9) Фильм потрясающий! Что ни говори, а Питер Джексон сделал все, чтобы при ОЧЕНЬ ограниченном экранном времени воссоздать дух трилогии Толкина! Мне кажется, ему это удалось! Игра актеров завораживает, одна из самых - самых удачных сцен — Саммат Наур, уничтожение Кольца параллельно с битвой у Мораннона. Вызывает практически те же и практически такие же сильные эмоции, как и книга — это высшая похвала! Вывод : Толкин — лучший писатель всех времен и народов! Питер Джексон — талантливый режиссер! Актеры — молодцы! =)

Правильно: 1

Предсказано: 2

- (10) Фильм заявлен как триллер, но уже после 15 минут просмотра осознаешь, что жанр точно указан не верно. Все происходящее на экране больше походит не на триллер, а на пародию с отклонениями в нелепую комедию. Не удивительно, что в зале все время просмотра вместо визгов ужаса стоял дикий ржач. Сценарий явно писался методом копи - паста из большого количества зарубежных источников, причем беря из них только набившие оскомину жанровые ходы. В итоге все просто до ужаса предсказуемо, где - то на минуту вперед, и ни капли не страшно. Финал вообще с легкостью угадывается чуть ли не в самом начале фильма и от того концовка еще более удручающа. Фобии

людей утрированы настолько, что опять же превращают все в комедию. Игра актеров вообще не понятна, скорее даже совсем не наблюдается. Особенно когда ужас в их глазах вызывает только смех, а запаздывание эмоций вообще добивает. Ну а реплики вроде « не нравится мне все это », « здесь живет зло », " я боюсь » похоже нужны, чтобы окончательно уморить зрителя. Звук, от которого должны трястись поджилки, местами настолько нелепый и никак не совместим с видео рядом. От того, что громкость резко возрастает каждые 30 секунд, атмосфернее и страшнее никак не становится. За все время просмотра не был увиден не один новый прием, поразивший зрителя, зато набор штампов превосходный, но выполнен он как - то слишком нелепо. Ляпы и недоработки наполняют фильм до краев, тут вам и кровь, летающая по замысловатой траектории в обход всех законов гравитации, и полтергейст, ползущий не только по стенам, и скелеты, на которых явно не хватило денег, и много чего еще несусветно смешного. Единственное ради чего « можно посмотреть разок » это реплики героя Петра Федорова, скорее всего многие « пойдут в народ ».

Правильно: 1

Предсказано: 0

- (11) О фильме : В целом фильм получился интересным, правда очень запутанным. Сначала всё понятно, но ближе к концу будто сам попадаешь туда и пытаешься узнать эту истину, которую ищет Ди Каприо. И не понятно, но очень интересно : что на самом деле происходит, а что ему, Маршалу, всего лишь кажется. И в итоге, фильм можно понять по разному , не говоря уж о странном конце, когда « напарник » назвал его Теди.. Будто всё и было запланировано ранее, заманить его. « Что лучше : жить монстром, или умереть человеком? ». На протяжении всего фильма он пытается найти ответы на все вопросы, которые появляются один за одним. А для зрителя ведь еще столько неразгаданных загадок. Плюсы : Интересный сюжет, довольно непредсказуемые последние полчаса фильма, отличная игра актеров. Минусы : Мне показался фильм слишком уж запутанным. Всем советую посмотреть.

Правильно: 2

Предсказано: 1

- (12) Как будто тандему сценаристов - продюсеров - режиссеров было мало откровенно поиздеваться над Сильвестром Сталлоне, выставив в издевательском свете его благородного Рокки Бальбоа, якобы принявшего сторону захватчиков — могущественных персов! В премьерный уик - энд « Знакомство со Спартаканцами », невзирая на традиционно разгромные рецензии и даже уничижительные отзывы простых зрителей, взлетело на вершину национального киночарта с результатом \$ 18, 5 млн., нахально потеснив « Рэмбо 4 ». В довершение ко всему долгожданный « сиквел » боевика о разгневанном « зелёном берете » в отставке потерпел сокрушительное поражение в России от « Самого лучшего фильма », испечённого явно по зельтцеровско - фридберговским рецептам. Прямо - таки не верится ... И лишь достаточно скромные итоговые показатели очередного детища неутомимого тандема, наверняка успевшего примерить на себя лавры главных пересмешников Голливуда самого начала XXI века, не позволяют окончательно отчаяться : когда - нибудь они успеют приесться. Хотя, положив руку на сердце, не без удивления отмечаешь, что от опуса к опусу (разумеется, без учёта « Очень страшного кино » / 2000 /, в большей степени являвшегося плодом фантазии братьев

Уэйнсов) результат у единомышленников понемногу улучшается. То, что для Мела Брукса (пояс, охраняющий девственность королевы Марго, даже является цитатой из его « Робина Гуда : мужиков в трико » / 1993 /), труппы « Монти Пайтон » или трио ZAZ могло лечь позорным пятном на всю жизнь, для Зельтцера и Фридберга кажется настоящим достижением. Да и чего скрывать, раз уж несколько моментов действительно способны вызвать пусть не смех до упаду, но ехидную усмешку — точно. Особенно забавен фрагмент с поочерёдной (« Это Спарта ... ») отправкой прямиком в пропасть — вслед за разбитным персидским посланцем — массы других назойливых личностей. Подозреваю, что кинематографисты помогли реализоваться давней мечте многих рядовых граждан США, не приученных существовать без телевизора, но вместе с тем — порядком подуставших от ежедневного сотворения кумира в том же « Американском идоле ». Нечто схожее можно заявить и по поводу отдельных индивидов : Америки Ферреры в привычном облики страшилки Бетти, « бритой на всю голову » Бритни Спирс и, конечно, пресловутой Пэрис Хилтон. А ещё удачен славный проход спартанцев, бодро распевających « I Will Survive

Правильно: 0

Предсказано: 1

2. В целом, модель часто ошибается на тэге 1, в том числе на тех примерах, разметка которых у нас не вызывает нареканий. Можно предположить, что модели сложнее уловить, как маркируется нейтральность, чем яркое положительное или отрицательное отношение, поскольку последние имеют больше эксплицитных отличительных черт на разных уровнях языка.
- (13) Есть одна фраза, высказанная свободным питерским художником : " Я бы этим безруким дизайнерам все руки бы поотростил », так вот и тут, только аж глаза разбегаются скольким людям и сколько всего « поотростить » надо. Начну, как уже стало доброй традицией у многих рецензентов, с предыстории. Итак, однажды, обуреваемая дикой скукой, я наткнулась на рекламу чудного фильма и решила, так уж и быть, потратить немного кровных (мне даже удалось уговорить на такую расточительность пару своих друзей) и отправиться в кино, посмотреть « Цветок дьявола ». Признаюсь честно, мы изначально шли на сию картину что бы весело провести время, хотя я и питала слабую надежду что быть может все не так уж плохо. Но, как заметила одна из рецензентов, наши планы посмеяться от души потерпели фиаско. Действительно становится скучно ... скучно и страшно. Страшно лишь от того, что люди, взрослые умные люди, выпуская на экран подобного рода фильмы, рассчитывают на некое признание, быть может одобрение и хорошие отзывы (про материальную сторону вопроса я вообще молчу), а еще, как правило, автор пытается донести до зрителя некую идею, тогда что же выходит, в голове у госпожи Гроховской некий странный и вовсе не съедобный винегрет (уж простите)? Мало того, что я не увидела никакой морали (даже намек на онную), так я еще и почерпала для себя много всего интересного. Так что теперь, когда мне приснится бредовый сон я пойду к гадалке и пускай устраивает темные ритуалы, как ведь, а вдруг я что важное упущу, или же если мне понадобится книга, датируемая примерно тринадцатым веком я знаю куда идти — конечно же в сельскую библиотеку, так есть все что угодно, достаточно лишь порыться в архивах! Однако, боюсь что описывая все « полезные заметки » из фильма могу

состариться, так что перейдем к главным героям. Полина — да, в ней определенно что-то есть, внешне, но по-моему мой миксер и то более живой и сообразительный, чем она. Тугодумие и аморфность — вот визитная карточка героини. И да, кто-нибудь мне объяснит какого дьявола нужно приходить на вечеринку, что бы пройти мимо всех и пойти на задний

Правильно: 0

Предсказано: 1

- (14) На самом деле меня фильм порадовал и на мой взгляд он получился во многом лучше, чем « обитаемый остров ». Сам я книгу Головачева не читал, но сюжет в фильме понятен и раскрыт. Фильм понравился тем, что : 1. — не разделен на отдельные части, все логически завершено, хотя продолжений тоже можно снять множество. 2 . — игра актеров на среднем уровне (но лучше чем в « Обитаемом острове ») 3. — Саундтрек очень порадовал. 4. — В фильме чувствуется атмосфера и тот , кто ее не увидел скорее всего просто пришел тупо посмотреть видеоряд, наличие в нем спецэффектов, не более (не считаю, что люди, ориентирующиеся только на спецэффекты, могут трезво оценить фильм).

Правильно: 2

Предсказано: 1

3. В датасете часто встречаются тексты, которые не похожи на отзывы, а представляют собой описание сюжета фильма без субъективной оценки. Такие примеры могут “сбивать” модель, потому что размечаются они как нейтральные (так как не несут оценки), но при этом в самом описании может доминировать позитивно или негативно окрашенная лексика, в зависимости от сюжета.

- (15) В старой Варшаве, среди сотен тысяч других поляков живет Пианист. Он красиво и стильно одет, сыт, гладко выбрит, сравнительно богат, имеет постоянную работу — играет музыку диковинной красоты для сотен тысяч других поляков, поклонников своего таланта. Его семья ни в чем не нуждается, и сам он ни в чем не нуждается, да и зачем? Все, что нужно, у него имеется, а какие-то излишества — не для него. В мире, где голод может заставить отнять еду у старой немощной старушки ; где таких же, как он, убивают просто так, для развлечения ; где денег едва хватает только на то, что бы поесть ; в мире, где царят нищета, голод и слезы — в зверском мире гитлеровского нацизма живет Пианист. В этом мире люди умирают прямо на улицах, от голода, холода, но еще в большей степени — от усталости и безысходности. В этом мире разрушаются семьи, нет, не из-за ссор и разногласий, совсем по другой причине. Одна из этих семей — семья Пианиста. Теперь он, оторванный от своих родных, вынужден работать на гитлеровцев, каждый день рискуя погибнуть ни за что. Но риск этот далеко не благородный, да и в общем - то, не своевольный. Время от времени появляются ниточки надежды, но так же быстро угасают. Теперь Пианист вынужден питаться сухими крупами, радоваться каждому куску хлеба или случайной банке огурцов, которую он смог найти в разрушенном доме. С течением времени, он понимает, что позиции немцев пошатнулись и верит, что все закончится хорошо. Один — грязный, небритый, голодный, больной, но искренне ждущий победы. В восстановленной, свободной Варшаве, среди сотен тысяч других поляков, живет Пианист. Он снова красиво и стильно одет, гладко выбрит и сыт. Он вернулся к своей любимой работе — снова играет музыку диковинной красоты, которая проникает прямо в сердце, полностью захватывая

его (сердце), не оставляя равнодушным даже самых избирательных ценителей фортепьянной музыки. Один, без семьи, хотя с друзьями и коллегами, он вернулся к той жизни, к которой привык. И теперь уже ничто не сможет поколебать его покоя, никакая сила не сможет отнять его любимую работу, никто не посмеет больше подвергнуть его тому, что он уже однажды пережил.

Только вот музыка

Правильно: 1

Предсказано: 2

- (16) Часто ли, в повседневной жизни, мы задумываемся о вечном ? Часто ли, в повседневной жизни, мы обращаем внимание на Красоту? Часто ли, в повседневной жизни, мы останавливаемся на миг и вспоминаем то, что есть более ценные вещи на Земле, чем то, что окружает нас каждый день? А что если жить остаётся всего несколько дней? Можно ли успеть за это, казалось бы, короткое время, открыть свою душу, чтобы достучаться до небес? А ведь за это время, можно исполнить свою мечту — одну, самую сокровенную, самую главную, на которую в обычной жизни нет времени. За это время, человек, о существовании которого день назад ты даже ещё не знал, может стать роднее, чем все , кто был до него. За это время, можно совершать безумства, на которые ты бы никогда не отважился. За это время, можно понять, для чего и зачем ты жил. За это время можно просто научиться жить ... И твоя душа обязательно достучится до небес. А тогда уже не страшно умирать.

Правильно: 1

Предсказано: 2

Вывод

Результаты, полученные на данном датасете не представляется возможным использовать для оценки применимости семантической разметки для обогащения эмбедингов, поскольку метрики, полученные на тестировании говорят о том, что модель не обучается – precision, recall, f1-score около 0.35 на всех трех классах говорит о том, что модель предсказывает на каждом классе только треть примеров правильно. Что касается задачи анализа тональности на примере полученных нами результатов, можно сделать вывод, что сложнее всего модель справляется с определением нейтральных комментариев.

Заключение

В ходе выполнения работы мы проанализировали основные, существующие на данный момент методы представления текстовых данных в машиночитаемом виде, рассмотрели их преимущества и недостатки, и остановили свой выбор на модели BERT, основанной на архитектуре трансформеров. Выбор был обусловлен тем, что данная модель показывает наилучшие результаты на задачах анализа текста на данный момент.

Мы рассмотрели различные форматы разметки семантической информации, учли их особенности и выбрали для разметки, на основе которой будет осуществляться обогащение семантической информацией формат разметки CoBaLD, семантический характеристики которого базируются на формате Compreno.

Мы также изучили способы обогащения эмбедингов языковых моделей, которые использовались исследователями ранее, и остановились на конкатенации с эмбедингами семантики, сформированными при помощи LSTM. Этот метод позволяет учитывать широкий спектр семантических различий, при этом легко встраивается в архитектуру для обучения модели.

В качестве датасетов для проведения исследования были выбраны датасеты, созданные для задач детекции токсичности и анализа тональности соответственно. Однако, в ходе исследования стало понятно, что датасет, выбранный нами для анализа тональности, непригоден для исследования ввиду низкого качества разметки. Кроме того, на основе анализа ошибок модели на данном датасете можно заметить, что хуже всего модель справляется с определением текстов с нейтральной тональностью.

Результаты, полученные после обучения и валидации модели для задачи детекции токсичности, показали, что обогащение модели семантической информацией в формате CoBaLD не влияет на качество решения данной задачи. Также анализ ошибок модели показал, что модель часто ошибается на слишком коротких данных, а также имеет тенденцию размечать, как токсичные, те тексты, в которых встречается obscene лексика.

Список литературы

1. Abend O., Rappoport A. Universal conceptual cognitive annotation (UCCA) //Proceedings of the 51st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). – 2013. – С. 228-238.
2. Baker C. F., Fillmore C. J., Lowe J. B. The berkeley framenet project //COLING 1998 Volume 1: The 17th International Conference on Computational Linguistics. – 1998.
3. Banarescu L. et al. Abstract meaning representation for sembanking //Proceedings of the 7th linguistic annotation workshop and interoperability with discourse. – 2013. – С. 178-186.
4. Boguslavsky I. M. et al. Constructing a Semantic Corpus for Russian: SemOntoCor //Proceedings of the International Conference “Dialogue. – 2023. – Т. 2023.
5. Bojanowski P. et al. Enriching word vectors with subword information //Transactions of the association for computational linguistics. – 2017. – Т. 5. – С. 135-146.
6. Chomsky N. Three models for the description of language //IRE Transactions on information theory. – 1956. – Т. 2. – №. 3. – С. 113-124.
7. Davidson. 1969. The individuation of events. In N. Rescher, editor, Essays in Honor of Carl G. Hempel. D. Reidel, Dordrecht.
8. Dementieva D. et al. Russe-2022: Findings of the first russian detoxification shared task based on parallel corpora //Computational Linguistics and Intellectual Technologies. – 2022.
9. Devlin J. et al. Bert: Pre-training of deep bidirectional transformers for language understanding //Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers). – 2019. – С. 4171-4186.
10. Fellbaum C. (ed.). WordNet: An electronic lexical database. – MIT press, 1998.
11. Ganitkevitch J., Van Durme B., Callison-Burch C. PPDB: The paraphrase database //Proceedings of the 2013 conference of the north american chapter of the association for computational linguistics: Human language technologies. – 2013. – С. 758-764.
12. Georgakopoulos S. V. et al. Convolutional neural networks for toxic comment classification //Proceedings of the 10th hellenic conference on artificial intelligence. – 2018. – С. 1-6.
13. Goodfellow I. et al. Deep learning. – Cambridge : MIT press, 2016. – Т. 1. – №. 2., стр. 131–133.
14. Гусева И. М. Обогащение эмбеддингов Word2Vec семантической информацией на базе формата CoBaLD // РГГУ. – 2024
15. Hajic J., Vidová-Hladká B., Pajas P. Theprague dependency treebank: Annotation structure and support //Proceedings of the IRCS workshop on linguistic databases. – Philadelphia : University of Pennsylvania, 2001. – С. 105-114.
16. Hochreiter S., Schmidhuber J. LSTM can solve hard long time lag problems //Advances in neural information processing systems. – 1996. – Т. 9.
17. Jigsaw. 2018. Toxic comment classification challenge. <https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge>. Accessed: 2025-06-26.
18. Jigsaw. 2019. Jigsaw unintended bias in toxicity classification. <https://www.kaggle.com/c/jigsaw-unintended-bias-in-toxicity-classification>. Accessed: 2025-06-26.

19. Jigsaw. 2020. Jigsaw multilingual toxic comment classification.
<https://www.kaggle.com/c/jigsaw-multilingual-toxic-comment-classification>.
Accessed: 2025-06-26.
20. Joulin A. et al. Fasttext. zip: Compressing text classification models //arXiv preprint arXiv:1612.03651. – 2016.
21. Jurafsky D., Martin J. H. Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition. – 2008. – C. 9-13
22. Jurafsky D., Martin J. H. Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition. – 2025. – 1-26
23. Kim J. K. et al. Intent detection using semantically enriched word embeddings //2016 IEEE spoken language technology workshop (SLT). – IEEE, 2016. – C. 414-419.
24. LeCun Y., Bengio Y., Hinton G. Deep learning //nature. – 2015. – T. 521. – №. 7553. – C. 436-444.
25. Lee M. Gelu activation function in deep learning: a comprehensive mathematical analysis and performance //arXiv preprint arXiv:2305.12073. – 2023.
26. Manning C. D. An introduction to information retrieval. – 2009. – 266-267
27. Marcus M., Santorini B., Marcinkiewicz M. A. Building a large annotated corpus of English: The Penn Treebank //Computational linguistics. – 1993. – T. 19. – №. 2. – C. 313-330.
28. Marie-Catherine De Marneffe, Bill MacCartney, Christopher D Manning, et al. 2006. Generating typed dependency parses from phrase structure parses. // Lrec, volume 6, P 449–454.
29. Mel'čuk I. Semantics: From Meaning to Text. Vol. 1. — Amsterdam/Philadelphia: John Benjamins. — 2012.
30. Mel'čuk I. Semantics: From Meaning to Text. Vol. 2. — Amsterdam/Philadelphia: John Benjamins. — 2013.
31. Mel'čuk I. Semantics: From Meaning to Text. Vol. 3. — Amsterdam/Philadelphia: John Benjamins. — 2015.
32. Mikolov T. et al. Efficient estimation of word representations in vector space //arXiv preprint arXiv:1301.3781. – 2013.
33. Palmer M., Gildea D., Kingsbury P. The proposition bank: An annotated corpus of semantic roles //Computational linguistics. – 2005. – T. 31. – №. 1. – C. 71-106.
34. Pennington J., Socher R., Manning C. D. Glove: Global vectors for word representation //Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP). – 2014. – C. 1532-1543.
35. Peters M. E. et al. Dissecting contextual word embeddings: Architecture and representation //arXiv preprint arXiv:1808.08949. – 2018.
36. Petrova M. A. The compreno semantic model: the universality problem //International Journal of Lexicography. – 2014. – T. 27. – №. 2. – C. 105-129.
37. Petr Sgall, Eva Hajico v a, and Jarmila Panevov' a. 1986. The Meaning of the Sentence and Its Semantic and Pragmatic Aspects. Academia/ReidelPublishing Company, Prague, Czech Republic/Dordrecht, Netherlands.
38. Pittaras N. et al. Text classification with semantically enriched word embeddings //Natural Language Engineering. – 2021. – T. 27. – №. 4. – C. 391-425.
39. Ribeiro F. N. et al. Sentibench-a benchmark comparison of state-of-the-practice sentiment analysis methods //EPJ Data Science. – 2016. – T. 5. – C. 1-29.

40. Robert M. W. Dixon. 2005. A Semantic Approach To English Grammar. Oxford University Press. Robert M. W. Dixon. 2010a. Basic Linguistic Theory: Methodology, volume 1. Oxford University Press.
41. Robert M. W. Dixon. 2010b. Basic Linguistic Theory: Grammatical Topics, volume 2. Oxford University Press.
42. Robert M. W. Dixon. 2012. Basic Linguistic Theory: Further Grammatical Topics, volume 3. Oxford University Press.
43. Rong X. word2vec parameter learning explained //arXiv preprint arXiv:1411.2738. – 2014.
44. Rundell M. Macmillan English Dictionary: The End of. – 2002.
45. Smetanin S. Toxic comments detection in Russian //Computational Linguistics and Intellectual Technologies: Proceedings of the International Conference “Dialogue. – 2020. – С. 17-20.
46. Spärck Jones K. A statistical interpretation of term specificity and its application in retrieval //Journal of documentation. – 2004. – Т. 60. – №. 5. – С. 493-502.
47. Vaswani A. et al. Attention is all you need //Advances in neural information processing systems. – 2017. – Т. 30.
48. Wankhade M., Rao A. C. S., Kulkarni C. A survey on sentiment analysis methods, applications, and challenges //Artificial Intelligence Review. – 2022. – Т. 55. – №. 7. – С. 5731-5780.
49. Zampieri M. et al. Semeval-2019 task 6: Identifying and categorizing offensive language in social media (offenseval) //arXiv preprint arXiv:1903.08983. – 2019.
50. Zhang A. et al. Dive into deep learning. – Cambridge University Press, 2023.
51. Zhang Z. et al. Semantics-aware BERT for language understanding //Proceedings of the AAAI conference on artificial intelligence. – 2020. – Т. 34. – №. 05. – С. 9628-9635.
52. Анисимович К. В. и др. Синтаксический и семантический парсер, основанный на лингвистических технологиях //Международная конференция по компьютерной лингвистике «Диалог». URL: <http://www.dialog-21.ru/digests/dialog2012/materials/pdf/Anisimovich.pdf> (дата обращения: 07.01.2024). – 2012.
53. Баяк И.С. Создание трёхуровневого парсера для формата Enhanced CoBaLD // Магистерская диссертация, МФТИ. – 2024
54. Гусева И. М. Обогащение эмбедингов Word2Vec семантической информацией на базе формата
55. Тищенко А. А. Обогащение эмбедингов больших языковых моделей семантической информацией на базе формата CoBaLD // РГГУ. – 2024